

IMPLEMENTASI PENDEKATAN *RECURRENT NEURAL NETWORK* UNTUK KLASIFIKASI EMOSI PADA TEKS *CHAT*

TUGAS AKHIR

Diajukan untuk melengkapi tugas-tugas dan memenuhi syarat-syarat guna memperoleh gelar Sarjana Komputer

Oleh:

M. NIZAR ASYKARY

2108107010043



**PROGRAM STUDI INFORMATIKA DEPARTEMEN INFORMATIKA
FAKULTAS MATEMATIKA DAN ILMU PENGETAHUAN ALAM
UNIVERSITAS SYIAH KUALA, BANDA ACEH
JULI, 2026**

PENGESAHAN

IMPLEMENTASI PENDEKATAN *RECURRENT NEURAL NETWORK* UNTUK KLASIFIKASI EMOSI PADA TEKS *CHAT*

Oleh:

Nama : M. Nizar Asykary
NPM : 2108107010043
Jurusan : Informatika

Menyetujui:

Pembimbing I,



Zahnur, S.Si, M.IT.

NIP. 196905291994031002

Pembimbing II,



Husaini M.Sc.

NIP. 198806242022031006

Mengetahui:

Koordinator Program Studi Informatika

Universitas Syiah Kuala,



Rasudin Abubakar, S.Si., M.Info.Tech.

NIP. 197410011999031001

ABSTRAK

Klasifikasi emosi pada teks *chat* merupakan salah satu penerapan *Natural Language Processing* (NLP) yang bertujuan untuk mengidentifikasi emosi yang terkandung dalam suatu teks secara otomatis. Penelitian ini bertujuan mengimplementasikan metode *Recurrent Neural Network* untuk mengklasifikasikan emosi pada teks berbahasa Inggris ke dalam empat kategori, yaitu *anger*, *fear*, *joy*, dan *sadness*. Sebelum proses pelatihan dilakukan, data melalui tahapan *preprocessing* yang meliputi *text cleaning*, *case folding*, *tokenization*, *stopword removal*, dan *stemming*. Selanjutnya dilakukan proses *label encoding*, *padding sequence*, dan *word embedding* sebagai representasi data sebelum diproses oleh model. Model dibangun menggunakan arsitektur *Embedding*, *Simple RNN*, dan *Dense* dengan *optimizer* Adam serta mekanisme *Early Stopping*. Pengujian dilakukan menggunakan beberapa nilai *learning rate*, yaitu 0,0001, 0,0003, dan 0,0005. Hasil pengujian menunjukkan bahwa *learning rate* 0,0003 memberikan performa terbaik dengan akurasi sebesar 88%, sedangkan *learning rate* 0,0001 dan 0,0005 masing-masing menghasilkan akurasi sebesar 85%. Selain memberikan akurasi yang lebih tinggi, model dengan *learning rate* 0,0003 juga menghasilkan prediksi yang lebih konsisten pada implementasi aplikasi berbasis Streamlit. Analisis terhadap panjang teks menunjukkan bahwa kategori *Short* memperoleh akurasi 94,49%, *Medium* 89,76%, dan *Long* 78,66%, sehingga panjang teks memengaruhi tingkat akurasi klasifikasi. Hasil pengujian juga menunjukkan bahwa keberadaan kata-kata tertentu dalam suatu kalimat memiliki kecenderungan untuk memengaruhi hasil prediksi emosi, sehingga pemilihan kosakata menjadi salah satu faktor yang berperan dalam proses klasifikasi yang dilakukan oleh model. Model yang dihasilkan kemudian diimplementasikan ke dalam aplikasi berbasis Streamlit untuk melakukan klasifikasi emosi secara langsung. Hasil penelitian menunjukkan bahwa metode RNN mampu mengklasifikasikan emosi pada teks *chat* dengan baik, termasuk pada kalimat yang memiliki struktur penulisan yang tidak teratur.

Kata kunci: *Recurrent Neural Network*, Klasifikasi Emosi, *Natural Language Processing*, Streamlit

ABSTRACT

Emotion classification in chat text is one of the applications of Natural Language Processing (NLP) that aims to automatically identify emotions expressed in text. This study aims to implement the Recurrent Neural Network method to classify English text into four emotion categories: anger, fear, joy, and sadness. Before the training process, the data underwent preprocessing stages including text cleaning, case folding, tokenization, stopword removal, and stemming. Subsequently, label encoding, padding sequence, and word embedding were applied as data representation techniques before being processed by the model. The proposed model was built using an Embedding layer, a Simple RNN layer, and a Dense layer with the Adam optimizer and an Early Stopping mechanism. The model was evaluated using three different learning rates: 0.0001, 0.0003, and 0.0005. The experimental results showed that a learning rate of 0.0003 achieved the best performance with an accuracy of 88%, while learning rates of 0.0001 and 0.0005 each achieved an accuracy of 85%. In addition to achieving the highest accuracy, the model with a learning rate of 0.0003 also produced more consistent predictions when implemented in the Streamlit-based application. Furthermore, text length analysis revealed classification accuracies of 94.49% for the Short category, 89.76% for the Medium category, and 78.66% for the Long category, indicating that text length influences classification performance. The experimental results indicate that the presence of certain words within a sentence tends to influence the predicted emotion, suggesting that word choice is one of the factors affecting the classification performed by the model. The trained model was subsequently implemented into a Streamlit-based web application for real-time emotion classification. The results demonstrate that the RNN method is capable of classifying emotions in chat text effectively, including texts with irregular sentence structures.

Keywords: Recurrent Neural Network, Emotion Classification, Natural Language Processing, Streamlit

KATA PENGANTAR

Segala puji dan syukur kehadiran Allah SWT yang telah melimpahkan rahmat dan hidayah-Nya kepada kita semua, sehingga penulis dapat menyelesaikan penulisan tugas akhir yang berjudul "**Implementasi Pendekatan *Recurrent Neural Network* Untuk Klasifikasi Emosi Pada Teks *Chat***" yang telah dapat diselesaikan sesuai rencana. Penulis banyak mendapatkan berbagai pengarahan, bimbingan, dan bantuan dari berbagai pihak. Oleh karena itu, melalui tulisan ini penulis mengucapkan rasa terima kasih kepada:

1. Kamaruddin Gani dan Zaidar sebagai kedua orang tua penulis, yang senantiasa memberikan dukungan penuh terhadap aktivitas dan kegiatan yang dilakukan penulis, baik secara moral maupun material, serta menjadi motivasi terbesar bagi penulis untuk menyelesaikan tugas akhir ini.
2. Bapak Prof. Dr. rer. nat. Ilham Maulana, S.Si., sebagai Dekan Fakultas MIPA Universitas Syiah Kuala.
3. Bapak Dr. Nizamuddin, M.Info.Sc., sebagai Ketua Departemen Informatika Fakultas MIPA Universitas Syiah Kuala.
4. Bapak Rasudin Abubakar, S.Si., M.Info.Tech. sebagai Koordinator Program Studi Informatika Fakultas MIPA Universitas Syiah Kuala.
5. Bapak Zahnur, S.Si, M.Info Tech., selaku Dosen Pembimbing I dan Dosen Wali yang telah memberikan bimbingan dan arahan kepada penulis, sehingga penulis dapat menyelesaikan Tugas Akhir ini.
6. Bapak Husaini, S.ST., M.Sc., selaku Dosen Pembimbing II yang telah memberikan bimbingan dan arahan kepada penulis, sehingga penulis dapat menyelesaikan Tugas Akhir ini.
7. Seluruh Dosen dan Staf di Departemen Informatika Fakultas MIPA, atas ilmu dan didikan nya selama perkuliahan.

8. M. Alfian Septian Nufi, Fagih Akram, T. Rifal Aulia, Furqan Ramadhan, dan seluruh anggota grup MASBROO yang senantiasa telah menemani, membantu, dan memberikan semangat serta dukungan kepada penulis selama masa perkuliahan di Departemen Informatika USK.

9. Sahabat dan teman-teman seperjuangan dari Departemen Informatika USK 2021 lainnya, yang telah memberikan dukungan moral yang luar biasa selama penulis menempuh pendidikan di Departemen Informatika USK.

Penulis menyadari adanya kekurangan dalam tulisan ini, baik dari aspek materi, metode, maupun bahasa yang digunakan. Harapan penulis semoga tulisan ini dapat memberikan manfaat bagi banyak orang dan berkontribusi pada kemajuan ilmu pengetahuan.

Banda Aceh, 28 Juli 2026

Muhammad Nizar Asykary
2108107010043

DAFTAR ISI

	Halaman
Halaman Judul	i
Halaman Pengesahan	ii
Pernyataan Bebas Plagiasi.....	iii
Surat Pernyataan	iv
Abstrak	v
<i>Abstract</i>	vi
Kata Pengantar.....	vii
Daftar Isi	ix
Daftar Gambar.....	xii
Daftar Tabel.....	xiii
Daftar Lampiran	xiv
BAB I PENDAHULUAN	1
1.1 Latar Belakang	1
1.2 Rumusan Masalah	3
1.3 Tujuan Penelitian	3
1.4 Manfaat Penelitian	4
BAB II TINJAUAN PUSTAKA	5
2.1 <i>Recurrent Neural Network</i>	5
2.1.1 <i>Gated recurrent unit</i>	6
2.1.2 <i>Backpropagation through time</i>	7
2.2 <i>Natural Language Processing</i>	7
2.3 Dataset.....	8
2.4 <i>Preprocessing</i>	8
2.4.1 <i>Text cleaning</i>	8
2.4.2 <i>Tokenization dan stopwords removal</i>	9
2.4.3 <i>Case folding</i>	9
2.4.4 <i>Stemming</i>	9
2.4.5 <i>Label encoding</i>	9
2.4.6 <i>Word Embedding</i>	10
2.4.7 <i>Padding Sequence</i>	10
2.5 Metrik Evaluasi	10

2.5.1	<i>Accuracy</i>	11
2.5.2	<i>Precision</i>	12
2.5.3	<i>Recall</i>	12
2.5.4	<i>F1 score</i>	12
2.6	Streamlit	13
2.7	Penelitian Terkait.....	14
BAB III	METODOLOGI PENELITIAN.....	16
3.1	Waktu Dan Lokasi Penelitian.....	16
3.2	Alat dan Bahan	16
3.3	Dataset.....	16
3.4	Metode Penelitian.....	17
3.4.1	Identifikasi Masalah	17
3.4.2	Studi Literatur	18
3.4.3	Pengumpulan Data	18
3.4.4	<i>Preprocessing</i> dan representasi data	18
3.4.5	Pemodelan Data	21
3.4.6	Pelatihan Model	22
3.4.7	Evaluasi Hasil	22
3.4.8	Visualisasi	23
BAB IV	HASIL DAN PEMBAHASAN.....	24
4.1	Pengumpulan Data	24
4.1.1	Distribusi dataset	25
4.2	<i>Preprocessing</i> Data.....	25
4.3	Pembentukan dan Representasi Data.....	27
4.4	Implementasi Model	28
4.4.1	Arsitektur model	28
4.4.2	Konfigurasi hyperparameter.....	29
4.4.3	Proses pelatihan model.....	31
4.5	Hasil Pelatihan Model.....	32
4.6	Evaluasi Model.....	34
4.7	Analisis Pengaruh Panjang Teks	36
4.8	Analisis Pengaruh <i>Keyword</i>	37
4.8.1	<i>Keyword</i> emosi.....	38
4.9	Implementasi Sistem.....	38

BAB V KESIMPULAN DAN SARAN	43
5.1 Kesimpulan	43
5.2 Saran	44
DAFTAR PUSTAKA.....	46
LAMPIRAN.....	49

DAFTAR GAMBAR

	<i>Halaman</i>
Gambar 2.1	Diagram RNN (Amazon Web Services, 2024) 5
Gambar 2.2	Struktur GRU (DataCamp, 2023)..... 6
Gambar 2.3	<i>Multiclass confusion matrix</i> (Medium, 2024).....11
Gambar 2.4	Tampilan streamlit13
Gambar 3.1	Tahapan penelitian17
Gambar 3.2	Visualisasi streamlit.....23
Gambar 4.1	Arsitektur model.....29
Gambar 4.2	Proses pelatihan model.....31
Gambar 4.3	Grafik <i>accuracy</i> model.....32
Gambar 4.4	Grafik <i>loss</i> model.....33
Gambar 4.5	<i>Confusion matrix</i>35
Gambar 4.6	Perbandingan akurasi37
Gambar 4.7	Halaman utama web.....39
Gambar 4.8	Hasil prediksi emosi.....40
Gambar 4.9	Visualisasi probabilitas emosi41
Gambar 4.10	Hasil <i>preprocessing</i> teks42

DAFTAR TABEL

	<i>Halaman</i>
Tabel 3.1 Proses <i>text cleaning</i>	19
Tabel 3.2 Proses <i>tokenization</i>	19
Tabel 3.3 Proses <i>case folding</i>	20
Tabel 3.4 Proses <i>Stopword Removal</i>	20
Tabel 3.5 Proses <i>Stemming</i>	20
Tabel 3.6 Proses <i>label encoding</i>	21
Tabel 3.7 Proses <i>padding sequence</i>	21
Tabel 3.8 Proses <i>embedding, hidden dan output layer</i>	22
Tabel 4.1 Struktur dataset.....	24
Tabel 4.2 Hasil pemeriksaan missing value	24
Tabel 4.3 Distribusi data pada setiap kelas emosi.....	25
Tabel 4.4 Proses <i>text cleaning</i>	26
Tabel 4.5 Proses <i>case folding</i>	26
Tabel 4.6 Proses <i>tokenization</i>	26
Tabel 4.7 Proses <i>stopword removal</i>	27
Tabel 4.8 Proses <i>stemming</i>	27
Tabel 4.9 Hasil <i>label encoding</i>	28
Tabel 4.10 Contoh hasil <i>tokenizer</i>	28
Tabel 4.11 Contoh hasil <i>padding sequence</i>	28
Tabel 4.12 Struktur arsitektur model.....	29
Tabel 4.13 Hyperparameter model	30
Tabel 4.14 Ringkasan hasil penelitian.....	31
Tabel 4.15 Hasil pelatihan model	33
Tabel 4.16 Hasil <i>classification report</i>	34
Tabel 4.17 Distribusi data berdasarkan panjang teks	36
Tabel 4.18 Akurasi data berdasarkan panjang teks	36
Tabel 4.19 <i>Keyword</i> emosi	38

DAFTAR LAMPIRAN

	<i>Halaman</i>
Lampiran 1. Kode Program <i>Import Library</i> dan Pembacaan Dataset.....	49
Lampiran 2. Kode Program <i>Preprocessing Data</i>	50
Lampiran 3. Kode Program Pembentukan <i>Word Embedding</i> dan <i>Label Encoding</i> .	51
Lampiran 4. Kode Program <i>Tokenizer</i> dan <i>Padding Sequence</i>	53
Lampiran 5. Kode Program Pembangunan dan Pelatihan Model.....	54
Lampiran 6. Kode Program Pengujian dan Evaluasi Model	55
Lampiran 7. Kode Program Analisis Pengaruh Panjang Teks	56
Lampiran 8. Kode Program Implementasi Streamlit	58

BAB I

PENDAHULUAN

1.1 LATAR BELAKANG

Komunikasi melalui teks sudah menjadi salah satu bentuk interaksi manusia yang paling dominan. Berbagai platform seperti media sosial, aplikasi pesan instan, dan forum daring semakin mempermudah interaksi antar individu. Melalui komunikasi berbasis teks, seseorang tidak hanya menyampaikan informasi, tetapi juga emosi dan perasaan mereka. Pemahaman emosi dalam teks menjadi semakin penting, terutama dalam berbagai bidang seperti layanan pelanggan, analisis tren media sosial, serta pemantauan kondisi psikologis seseorang (Zhou *et al.*, 2023).

Mengidentifikasi emosi dalam teks bukanlah hal yang sederhana. Makna sebuah kalimat sering kali bergantung pada konteks, pilihan kata, dan bahkan gaya penulisan yang digunakan oleh seseorang. Kata yang sama dapat memiliki arti yang berbeda tergantung pada bagaimana dan dalam situasi apa kata tersebut digunakan, yang membuat analisis emosi dalam teks menjadi tantangan tersendiri yang memerlukan pendekatan yang lebih kompleks dibandingkan dengan sekadar pemrosesan teks biasa (Sharma *et al.*, 2022).

Dalam beberapa tahun terakhir, pendekatan *deep learning* seperti *Recurrent Neural Network* banyak diterapkan untuk mendeteksi emosi berdasarkan teks karena kemampuannya dalam memahami konteks urutan kata secara dinamis. RNN memanfaatkan struktur berulang untuk memproses data sekuensial, sehingga lebih unggul dibanding metode konvensional dalam mengenali pola bahasa yang merepresentasikan emosi manusia. Penelitian (Kaur *et al.*, 2023) menunjukkan bahwa penerapan arsitektur RNN dan variannya mampu meningkatkan performa klasifikasi emosi teks pada berbagai metrik evaluasi, seperti akurasi, presisi, dan *F1-score*, terutama ketika diimplementasikan pada data percakapan digital.

Pendekatan RNN terbukti efektif dalam berbagai tugas klasifikasi emosi, tetapi tantangan masih muncul ketika model dihadapkan pada teks yang tidak terstruktur seperti bahasa percakapan di media sosial atau aplikasi *chat*. Teks semacam ini sering kali mengandung bentuk ekspresi yang tidak baku,

penggunaan slang, serta struktur kalimat yang tidak mengikuti kaidah tata bahasa formal, sehingga dapat memengaruhi hasil klasifikasi. Perbedaan struktur dan gaya bahasa pada teks informal menjadi salah satu faktor utama yang menurunkan performa model *deep learning* dalam mengenali emosi secara akurat (Chutia et al., 2024).

Menurut (Mienye et al., 2024), RNN memiliki kemampuan untuk memproses data sekuensial dengan mempertahankan informasi dari langkah sebelumnya melalui mekanisme *hidden state*. Kemampuan ini memungkinkan RNN memahami keterkaitan antar kata dalam suatu urutan teks sehingga banyak digunakan pada berbagai tugas NLP, termasuk klasifikasi emosi.

Beberapa penelitian mencoba menggabungkan pendekatan RNN dengan *Convolutional Neural Network* (CNN) untuk meningkatkan akurasi dalam memahami konteks emosi dalam sebuah percakapan. Pendekatan ini tidak hanya mengandalkan teks, tetapi juga memasukkan fitur *valence-arousal* agar sistem dapat mengenali intensitas emosi dalam suatu teks, yang kemudian diterapkan dalam sistem *chatbot* agar respons yang diberikan lebih sesuai dengan konteks emosi penggunaannya (Chang and Hsing, 2021).

Penelitian mengenai klasifikasi emosi pada teks berbahasa Inggris juga telah banyak dilakukan karena ketersediaan dataset beranotasi yang mendukung pengembangan model NLP. Salah satu penelitian (Demszky et al, 2020) memperkenalkan *GoEmotions*, yaitu dataset berbahasa Inggris yang terdiri atas sekitar 58.000 komentar Reddit dengan 27 kategori emosi. Penelitian tersebut menunjukkan bahwa dataset berbahasa Inggris yang memiliki kualitas anotasi yang baik dapat menjadi dasar dalam pengembangan dan evaluasi berbagai model klasifikasi emosi, sehingga mendorong semakin berkembangnya penelitian di bidang pengenalan emosi berbasis teks.

Panjang teks merupakan salah satu faktor penting yang dapat memengaruhi performa model klasifikasi teks, termasuk dalam tugas klasifikasi emosi. Teks yang terlalu pendek sering kali tidak mampu memberikan cukup konteks bagi model untuk memahami emosi yang terkandung di dalamnya. Kalimat dengan jumlah kata sedikit cenderung hanya merepresentasikan potongan perasaan tanpa informasi pendukung,

sehingga model kesulitan menangkap pola yang jelas. Sebaliknya, teks yang terlalu panjang dapat menghasilkan *noise* atau informasi tambahan yang tidak relevan. Hal tersebut menyebabkan sistem harus memproses lebih banyak kata, yang berpotensi menurunkan fokus terhadap emosi dominan yang sebenarnya ingin diklasifikasikan. Panjang teks perlu diperhatikan dalam perancangan sistem klasifikasi emosi, karena perbedaan panjang *input* dapat menghasilkan variasi tingkat akurasi yang cukup signifikan (Kowsari *et al.*, 2019).

Penelitian penulis berfokus pada penerapan pendekatan RNN dalam klasifikasi emosi pada teks *chat*. Dengan pendekatan ini, diharapkan sistem dapat lebih memahami dan mengklasifikasikan emosi pengguna dengan akurat. Teknologi ini dapat diterapkan dalam berbagai bidang, seperti *chatbot* layanan pelanggan, analisis opini di media sosial, serta sistem pemantauan kondisi emosi dalam aplikasi berbasis kesehatan mental.

1.2 RUMUSAN MASALAH

Berdasarkan latar belakang yang telah dijelaskan sebelumnya, penelitian ini merumuskan beberapa permasalahan sebagai berikut:

1. Dibutuhkan implementasi pendekatan *Recurrent Neural Network* untuk klasifikasi emosi pada teks *chat*.
2. Dibutuhkan identifikasi terkait tingkat keakuratan *Recurrent Neural Network* dalam mengenali emosi pada teks *chat* yang memiliki kalimat tidak terstruktur.
3. Dibutuhkan analisis terkait pengaruh panjang teks terhadap akurasi dalam mengklasifikasikan emosi.

1.3 TUJUAN PENELITIAN

Berdasarkan rumusan masalah yang telah dikemukakan, penelitian ini memiliki beberapa tujuan sebagai berikut:

1. Mengimplementasikan pendekatan *Recurrent Neural Network* dalam klasifikasi emosi pada teks *chat*.
2. Melakukan identifikasi pada tingkat keakuratan pendekatan *Recurrent Neural Network* dalam mengenali emosi pada teks *chat* yang memiliki

kalimat tidak terstruktur.

3. Menganalisis pengaruh panjang teks terhadap akurasi dalam melakukan klasifikasi emosi.

1.4 MANFAAT PENELITIAN

Penelitian ini diharapkan dapat berkontribusi dalam pengembangan RNN untuk memahami emosi pada teks *chat*, sehingga dapat memperluas kajian di bidang pemrosesan bahasa alami. Selain itu, hasil penelitian ini diharapkan mampu memberikan dasar bagi pengembangan aplikasi praktis, seperti sistem *chatbot* yang lebih empatik atau platform analisis opini publik, yang dapat menyesuaikan respons sesuai dengan emosi pengguna.

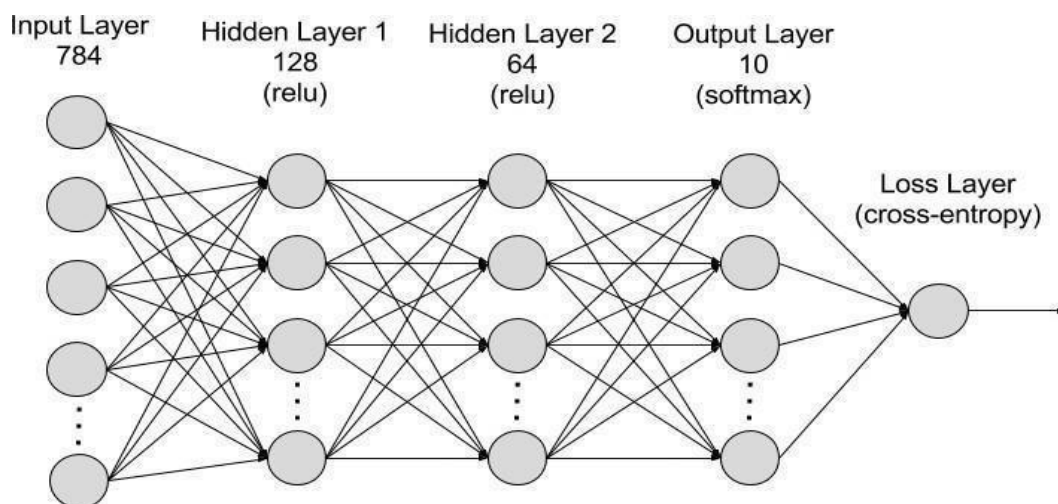
BAB II

TINJAUAN PUSTAKA

2.1 RECURRENT NEURAL NETWORK

Recurrent Neural Network adalah arsitektur jaringan saraf tiruan yang dirancang untuk mengolah data berurutan. Tidak seperti *feedforward neural network* yang hanya memproses *input* dalam satu arah, RNN memiliki koneksi umpan balik yang memungkinkan informasi dari *input* sebelumnya digunakan kembali untuk memengaruhi pemrosesan *input* berikutnya. Hal tersebut menjadikan RNN sangat cocok untuk menangani data teks, suara, maupun sinyal yang bersifat sekuensial (Brownlee, 2021).

RNN terdiri dari tiga lapisan utama, yaitu *input*, *hidden*, dan *output layer*. Pada setiap langkah waktu, *hidden layer* menerima masukan baru sekaligus membawa informasi dari langkah sebelumnya melalui mekanisme *hidden state*. Proses ini membentuk rantai komputasi yang memungkinkan *Gated Recurrent Unit* (GRU) memahami keterkaitan antar kata dalam sebuah kalimat. Keunikan utama arsitektur RNN adalah penggunaan bobot yang sama pada setiap langkah waktu, sehingga dapat mempelajari pola urutan tanpa memerlukan parameter baru di setiap tahap (Cho *et al.*, 2014).

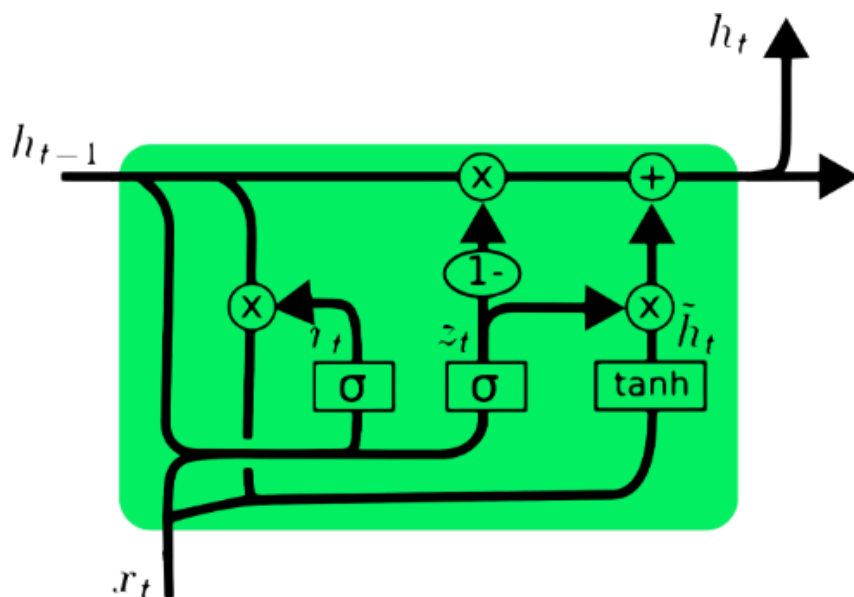


Gambar 2.1. Diagram RNN (Amazon Web Services, 2024)

Gambar 2.1 menunjukkan alur dasar RNN yang terdiri dari tiga komponen utama: *input layer*, *hidden layer*, dan *output layer*. Data teks masuk melalui *input layer*, lalu diproses pada *hidden layer* yang memiliki hubungan berulang (*recurrent connection*) untuk menyimpan informasi dari waktu sebelumnya, kemudian hasilnya diteruskan ke *output layer*.

2.1.1 Gated recurrent unit

Gated Recurrent Unit (GRU) merupakan varian dari RNN yang dikembangkan untuk mengatasi masalah *vanishing gradient* dan *exploding gradient* pada urutan panjang. GRU menggunakan dua gerbang utama yaitu *update gate* dan *reset gate*, dimana *update gate* mengatur seberapa banyak informasi dari masa lalu yang dibawa ke masa kini, dan *reset gate* menentukan seberapa banyak informasi lama yang diabaikan. Dengan struktur yang lebih sederhana dibandingkan *Long Short-Term Memory (LSTM)*, GRU mampu mempelajari ketergantungan jangka panjang secara lebih efisien dengan kebutuhan komputasi yang relatif lebih rendah (Cho et al., 2014).



Gambar 2.2. Struktur GRU (DataCamp, 2023)

Gambar 2.2 memperlihatkan dua gerbang utama, yaitu *update gate* dan *reset gate*. *Update gate* berfungsi mengatur seberapa banyak informasi lama yang dipertahankan, sedangkan *reset gate* menentukan seberapa banyak informasi lama yang dilupakan saat memproses *input* baru. Dengan desain yang lebih sederhana dibanding LSTM, GRU dapat belajar dependensi jangka panjang dengan efisien dan membutuhkan komputasi lebih ringan.

2.1.2 *Backpropagation through time*

Backpropagation through time (BPTT) adalah teknik pelatihan yang diperluas dari algoritma *backpropagation* untuk diterapkan pada RNN. Proses BPTT melibatkan *unrolling* atau pelebaran struktur sepanjang setiap langkah waktu dan menjumlahkan gradien dari setiap langkah tersebut untuk memperbarui bobot secara kolektif. Pendekatan ini memungkinkan RNN mempelajari ketergantungan urutan (*temporal dependencies*) dalam data sekuensial seperti teks atau sinyal waktu (QuarkML, 2023).

Secara singkat, BPTT bekerja dengan dua fase utama: pertama, *forward pass* di mana *input* urutan diproses dari langkah waktu awal hingga akhir; kedua, *backward pass* di mana kesalahan dihitung mulai dari akhir urutan dan merambat mundur ke langkah awal menggunakan aturan rantai (*chain rule*). Gradien untuk setiap parameter kemudian dijumlahkan dari semua langkah waktu yang dilalui. Metode ini menjadi inti dari pelatihan RNN dan varian terbaru seperti LSTM atau GRU meskipun detail teknis seperti *truncated BPTT* juga digunakan untuk efisiensi komputasi (Polivoda, 2021).

2.2 *NATURAL LANGUAGE PROCESSING*

Natural Language Processing (NLP) merupakan cabang kecerdasan buatan yang berfokus pada interaksi antara bahasa manusia dan komputer. Dalam klasifikasi emosi, NLP menyediakan teknik untuk mengubah teks mentah menjadi representasi numerik yang dapat dipahami komputer, meliputi tokenisasi, *word embedding*, serta pemodelan dengan jaringan saraf. Lewat teknik NLP sistem bisa mengenali pola linguistik

emosional seperti kata “senang” terkait dengan *joy*, atau “marah” dan “kesal” terkait dengan *anger* (Munezero *et al.*, 2014).

Klasifikasi emosi merupakan proses pengelompokan teks ke dalam kategori emosi tertentu seperti *joy*, *sadness*, *anger*, atau *fear*. Tujuan dari klasifikasi emosi adalah memahami aspek afektif dari komunikasi manusia, agar sistem komputer dapat mengenali dan merespons perasaan yang tersembunyi dalam teks—sangat penting dalam aplikasi seperti analisis opini publik atau pengembangan *chatbot* yang lebih empatik (Poria *et al.*, 2019).

2.3 DATASET

Dataset merupakan kumpulan data yang digunakan sebagai sumber informasi dalam proses pelatihan dan pengujian model klasifikasi emosi. Dataset pada penelitian berbasis teks biasanya terdiri atas kalimat atau percakapan yang sudah diberi label emosi, seperti *joy*, *sadness*, *anger*, dan *fear*. Dengan adanya label, model dapat belajar mengenali pola linguistik yang merepresentasikan emosi tertentu. Dataset yang baik umumnya memiliki distribusi data yang seimbang di setiap kelas, serta mengandung variasi gaya bahasa, baik formal maupun nonformal. Hal ini penting agar model mampu melakukan generalisasi dengan lebih baik terhadap data baru (Demszky *et al.*, 2020).

2.4 PREPROCESSING DAN REPRESENTASI DATA

Preprocessing data dapat didefinisikan sebagai tahap pengolahan awal terhadap data mentah agar siap digunakan dalam proses pelatihan model. Tahapan ini penting karena teks yang diambil dari percakapan sehari-hari sering kali memiliki bentuk yang tidak terstruktur, misalnya terdapat tanda baca, singkatan, atau emotikon. Proses *preprocessing* meliputi pemisahan kata, pembersihan teks, penghapusan kata umum yang tidak relevan, representasi kata dalam bentuk vektor, hingga penyamaan panjang urutan kata. Tahapan ini mampu meningkatkan kinerja model analisis emosi pada teks informal (Sailunaz *et al.*, 2018).

2.4.1 Text cleaning

Tahap awal dalam analisis emosi berbasis teks adalah pembersihan data. Proses ini mencakup penghapusan tanda baca, simbol, URL, angka, maupun karakter yang tidak relevan. Tujuannya adalah untuk menghasilkan data yang lebih konsisten dan dapat diproses oleh model dengan baik (Krouska *et al.*, 2017).

2.4.2 Tokenization dan stopwords removal

Tokenization merupakan proses pemisahan teks menjadi unit kata atau frasa, sedangkan *stopword removal* adalah langkah menghapus kata-kata umum yang tidak memberikan kontribusi besar terhadap klasifikasi, seperti “the”, “and” “of”, atau “is”. Kedua tahap ini mempersiapkan data agar lebih bermakna bagi proses pembelajaran mesin (Manning *et al.*, 2008).

2.4.3 Case folding

Case folding merupakan salah satu tahapan *text preprocessing* yang bertujuan untuk menyeragamkan representasi teks dengan mengubah seluruh huruf menjadi huruf kecil (*lowercase*) serta menghilangkan karakter yang tidak diperlukan, seperti angka dan tanda baca. Proses ini dilakukan untuk mengurangi variasi penulisan kata sehingga data menjadi lebih konsisten dan lebih mudah diproses pada tahap analisis berikutnya (Rahmi *et al.*, 2024).

2.4.4 Stemming

Stemming merupakan proses mengubah kata yang memiliki imbuhan menjadi bentuk dasar sehingga kata-kata yang memiliki makna serupa direpresentasikan dalam bentuk yang lebih konsisten. Tahap ini bertujuan mengurangi variasi kosakata sehingga dapat meningkatkan efektivitas proses klasifikasi teks dan membantu model dalam mengenali pola pada data (Rianto *et al.*, 2021).

2.4.5 Label encoding

Label encoding merupakan teknik yang digunakan untuk mengubah data kategorikal menjadi representasi numerik sehingga dapat diproses oleh algoritma pembelajaran mesin. Setiap kategori diberikan nilai numerik yang berbeda tanpa mengubah makna dari masing-masing kelas. Teknik ini banyak digunakan pada

proses klasifikasi untuk mengubah label target yang semula berbentuk teks menjadi nilai numerik sebelum dilakukan proses pelatihan model (Pargent et al., 2022).

2.4.6 *Word embedding*

Word embedding adalah metode representasi kata ke dalam bentuk vektor numerik sehingga kata-kata yang memiliki makna serupa memiliki representasi yang berdekatan. Metode populer seperti *Word2Vec*, *GloVe*, dan *FastText* banyak digunakan dalam penelitian NLP. Pada implementasi modern, *embedding layer* bawaan dari Keras atau TensorFlow juga sering dimanfaatkan untuk membangun representasi kata (Mikolov et al., 2013).

2.4.7 *Padding sequence*

Padding sequence merupakan teknik untuk menyamakan panjang urutan kata pada teks sebelum dimasukkan ke dalam model. Teks dalam bentuk percakapan memiliki panjang yang bervariasi, ada yang hanya satu kata, ada pula yang berbentuk paragraf singkat. Agar RNN dapat memproses semua *input* dengan ukuran yang konsisten, teks yang lebih pendek ditambahkan nilai nol hingga panjangnya sama dengan teks terpanjang dalam dataset. Dengan cara ini, setiap urutan kata dapat diolah secara efisien tanpa mengurangi informasi yang dimiliki (Kusal et al., 2023).

2.5 METRIK EVALUASI

Salah satu metrik evaluasi yang umum digunakan dalam tugas klasifikasi adalah *Multiclass confusion matrix*. *Multiclass confusion matrix* digunakan untuk mengevaluasi kinerja model klasifikasi yang memiliki lebih dari dua kelas. Matriks ini berbentuk tabel persegi 4x4 di mana setiap baris mewakili kelas aktual dan setiap kolom mewakili kelas prediksi. Nilai diagonal menunjukkan jumlah prediksi benar pada masing-masing kelas, sedangkan nilai di luar diagonal menggambarkan kesalahan klasifikasi ketika suatu kelas diprediksi sebagai kelas lain. Cara ini membuat peneliti dapat menganalisis pola kesalahan dan mengukur sejauh mana model mampu mengenali tiap kategori emosi secara seimbang (Pedregosa et al., 2011).

		PREDICTED classification				
		Classes	a	b	c	d
ACTUAL classification	a	TN	FP	TN	TN	
	b	FN	TP	FN	FN	
	c	TN	FP	TN	TN	
	d	TN	FP	TN	TN	

Gambar 2.3. *Multiclass confusion matrix* (Medium, 2024)

Gambar 2.3 menunjukkan bentuk *multiclass confusion matrix* yang digunakan untuk mengevaluasi model klasifikasi dengan lebih dari dua kelas. Setiap baris merepresentasikan kelas aktual, sedangkan setiap kolom menunjukkan kelas yang diprediksi oleh model. Sel-sel di dalam matriks berisi nilai *True Positive* (TP), *False Positive* (FP), *True Negative* (TN), dan *False Negative* (FN) yang menggambarkan hasil prediksi model untuk masing-masing kelas. Dengan struktur ini, dapat dilihat bagaimana model membedakan antar kelas (a, b, c, d) serta di mana kesalahan klasifikasi terjadi.

2.5.1 Accuracy

Accuracy menunjukkan proporsi prediksi yang benar terhadap seluruh data. Metrik ini sederhana dan umum dipakai, namun bisa menyesatkan pada dataset yang tidak seimbang (Rainio, 2024).

$$Accuracy = TP + TN / (TP + FP + FN + TN) \quad (2.1)$$

TP (*True Positive*) : Data positif yang diprediksi benar

TN (*True Negative*) : Data negatif yang diprediksi benar

FP (*False Positive*) : Data negatif yang salah diprediksi sebagai positif

FN (*False Negative*) : Data positif yang salah diprediksi sebagai negatif

2.5.2 *Precision*

Precision mengukur ketepatan prediksi positif. Nilai tinggi berarti model jarang salah saat mengklasifikasikan data sebagai positif (Hinojosa Lee, 2024).

$$Precision = TP / (TP + FP) \quad (2.2)$$

TP (*True Positive*) : Data positif yang diprediksi benar

FP (*False Positive*) : Data negatif yang salah diprediksi sebagai positif

2.5.3 *Recall*

Recall menilai sejauh mana model mampu menangkap semua data positif. *Recall* tinggi menunjukkan sedikit kasus positif yang terlewat (Rainio, 2024).

$$Recall = TP / (TP + FN) \quad (2.3)$$

TP (*True Positive*) : Data positif yang diprediksi benar

FN (*False Negative*) : Data positif yang salah diprediksi sebagai negatif

2.5.4 *F1 score*

F1-score merupakan rata-rata harmonik *precision* dan *recall*. Metrik ini lebih seimbang dibanding akurasi, terutama pada data yang tidak seimbang (Hinojosa Lee, 2024).

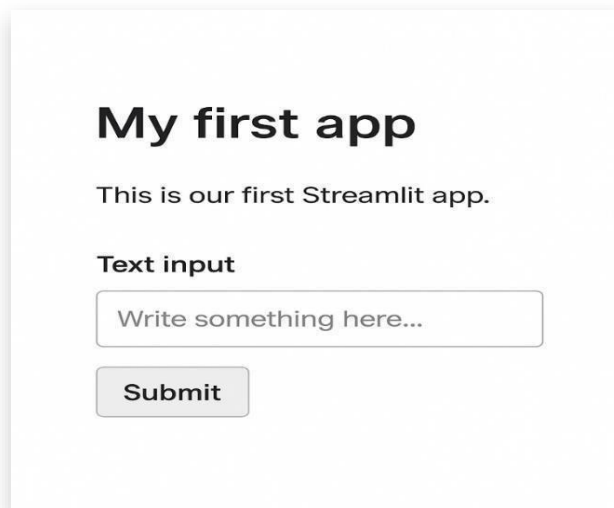
$$F1\ Score = (2 \times P \times R / (P + R)) \quad (2.4)$$

P : Nilai *Precision*

R : Nilai *Recall*

2.6 STREAMLIT

Streamlit merupakan sebuah *framework open-source* berbasis Python yang dirancang untuk mempermudah pengembangan aplikasi berbasis data. Dengan menggunakan Streamlit, model klasifikasi emosi dapat diintegrasikan ke dalam antarmuka sederhana yang interaktif. Pengguna cukup mengetikkan teks *chat* pada antarmuka aplikasi, kemudian sistem secara otomatis memproses teks tersebut dan menampilkan hasil prediksi emosi. Keunggulan utama Streamlit adalah kemampuannya menyajikan aplikasi *machine learning* secara cepat tanpa memerlukan penguasaan pengembangan web secara mendalam, sehingga hasil penelitian dapat diakses lebih luas dan praktis (McKinney, 2021).



Gambar 2.4. Tampilan streamlit

Antarmuka Streamlit diatas digambarkan sederhana dengan judul aplikasi, kolom *input* teks, dan tombol *submit*. Alur penggunaannya: pengguna mengetik teks ke dalam kolom *input*, lalu menekan tombol untuk mengirimkan data. Sistem yang sudah diintegrasikan dengan model klasifikasi emosi kemudian menampilkan hasil prediksi secara langsung. Keunggulan tampilan ini adalah kesederhanaan dan kemudahan penggunaan tanpa memerlukan penguasaan *web development* yang kompleks.

2.7 PENELITIAN TERKAIT

Penelitian (Zhang *et al.* 2018) membahas penggunaan RNN dalam analisis sentimen dan membuktikan efektivitas pendekatan ini dalam menangkap pola sekuensial pada teks. Mereka menjelaskan bahwa RNN mampu mengenali hubungan antar kata yang tersebar dalam urutan panjang, sehingga lebih unggul dibandingkan pendekatan tradisional berbasis *bag-of-words*. Penelitian ini juga menekankan pentingnya penggunaan *word embedding* untuk meningkatkan representasi kata, dan hasil eksperimen menunjukkan bahwa integrasi RNN dengan *embedding* modern secara signifikan meningkatkan akurasi analisis sentimen.

Penelitian lain (Demszky *et al.* 2020) memperkenalkan GoEmotions, sebuah dataset berlabel emosi yang berisi lebih dari 58 ribu komentar pengguna internet dari platform Reddit. Dataset ini terdiri dari 27 kategori emosi ditambah satu label netral, menjadikannya salah satu dataset paling komprehensif dalam penelitian klasifikasi emosi. Melalui serangkaian eksperimen, mereka menemukan bahwa model berbasis *transformer* seperti BERT mampu mencapai *F1-score* makro sebesar 0,46, menandakan adanya potensi besar namun juga menunjukkan bahwa klasifikasi emosi masih merupakan tantangan yang kompleks.

Penelitian lain (Hasan *et al.* 2022) menunjukkan bahwa varian RNN seperti GRU mampu memberikan hasil yang lebih efisien dibandingkan model standar karena strukturnya yang lebih sederhana. Dalam penelitian ini, GRU terbukti membutuhkan waktu pelatihan yang lebih singkat tanpa mengorbankan akurasi prediksi. Bahkan, dalam beberapa kasus, GRU menghasilkan performa yang lebih baik dibandingkan LSTM, terutama pada dataset dengan ukuran yang terbatas. Temuan ini memperkuat posisi GRU sebagai pilihan yang tepat dalam penelitian terkait klasifikasi emosi berbasis teks.

(Glenn *et al.*, 2023) melakukan penelitian mengenai klasifikasi emosi pada teks menggunakan metode *Bidirectional Long Short-Term Memory* (BiLSTM). Hasil penelitian menunjukkan bahwa model BiLSTM mampu mengidentifikasi emosi pada data teks dengan memanfaatkan informasi konteks dari arah maju dan mundur secara bersamaan. Perbedaan dengan penelitian yang

dilakukan penulis terletak pada metode yang digunakan, di mana penelitian ini menerapkan *Simple* RNN untuk mengklasifikasikan emosi pada teks *chat* ke dalam empat kategori emosi, yaitu *joy*, *sadness*, *anger*, dan *fear*.

Penelitian ini memilih Python sebagai bahasa pemrograman karena memiliki ekosistem yang kaya untuk *machine learning* dan *deep learning*. Python juga memiliki komunitas yang besar serta dokumentasi yang lengkap, yang dapat mempercepat proses pengembangan. Penggunaan Streamlit sebagai *framework* dikarenakan mampu menyediakan antarmuka aplikasi berbasis web yang sederhana namun interaktif, tanpa memerlukan keahlian yang mendalam pada proses pengembangan web.

BAB III

METODOLOGI PENELITIAN

3.1 WAKTU DAN LOKASI PENELITIAN

Penelitian dilakukan bertempat di Laboratorium Sistem Informasi dan Database Departemen Informatika Fakultas MIPA USK. Waktu yang dibutuhkan agar penelitian ini dapat diselesaikan adalah 6 bulan terhitung dari bulan September 2025 hingga Februari 2026.

3.2 ALAT DAN BAHAN

Alat dan Bahan yang akan digunakan pada penelitian ini terdiri dari beberapa perangkat keras (*hardware*) dan perangkat lunak (*software*) yang dijabarkan sebagai berikut:

1. Perangkat Keras

Laptop Lenovo ThinkPad T450s dengan RAM 8GB, Intel® Core™ i7-5600U, 2.60 GHz, Solid State Drive (SSD) 256GB.

2. Perangkat Lunak

Visual Studio Code 1.92.7, Python 3.10.6, Tensorflow 2.13.0, Keras 2.13.1, NumPy 1.24.3, Pandas 2.0.3, Matplotlib 3.7.2, Scikit-learn 1.3.0, NLTK 3.8.1, Seaborn 0.12.2, Streamlit 1.32.0.

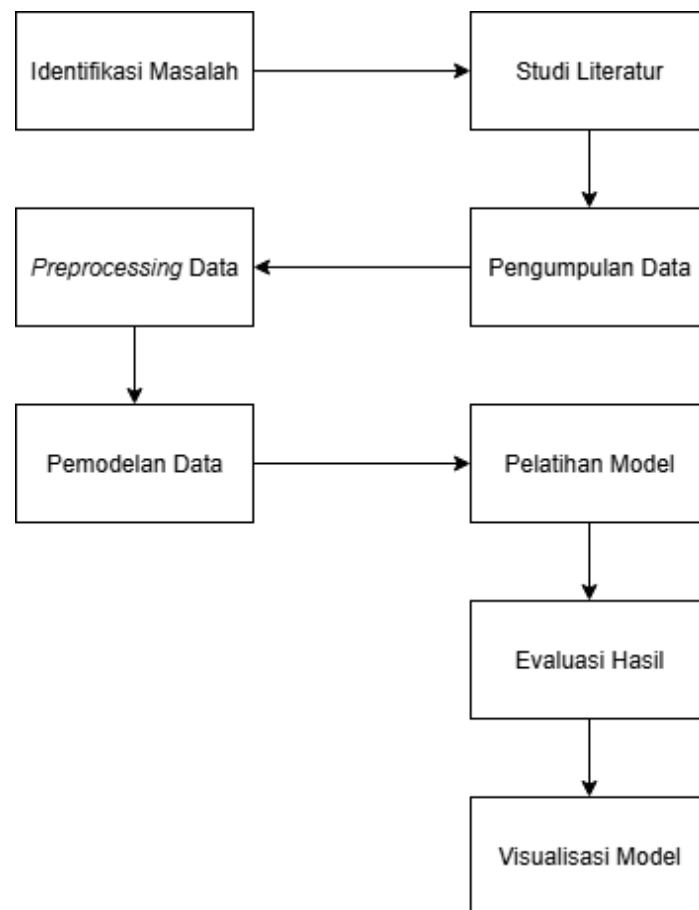
3.3 DATASET

Dataset yang digunakan dalam penelitian ini adalah *Emotions Dataset for NLP* yang diperoleh dari platform Kaggle. Dataset ini berisi kumpulan teks berbahasa Inggris yang telah diberi label emosi dan banyak digunakan dalam penelitian NLP, khususnya pada tugas klasifikasi emosi. Dataset terdiri dari empat kategori emosi, yaitu *sadness*, *joy*, *anger*, dan *fear*, dan telah dibagi ke dalam data latih (*training set*), data validasi (*validation set*), dan data uji (*test set*) untuk mendukung proses pelatihan dan evaluasi model klasifikasi emosi yang

dikembangkan dalam penelitian ini.

3.4 METODE PENELITIAN

Penelitian ini disusun melalui tahapan yang sistematis mulai dari identifikasi masalah, studi literatur, pengumpulan data, *preprocessing*, pemodelan, pelatihan model, evaluasi hasil, hingga visualisasi sederhana. Tahapan ini dapat digambarkan dalam bentuk alur penelitian sebagaimana ditunjukkan pada gambar 3.1 dibawah ini.



Gambar 3.1. Tahapan penelitian

3.4.1 Identifikasi masalah

Penelitian ini dimulai dengan mengidentifikasi permasalahan yang ada di media sosial, yaitu bagaimana teks *chat* yang bersifat tidak formal sering kali mengandung emosi yang sulit dikenali oleh sistem otomatis, sehingga menjadi

tantangan karena bahasa yang digunakan pada percakapan sehari-hari cenderung singkat, penuh variasi, dan kadang tidak mengikuti aturan baku. Diperlukan suatu metode berbasis *machine learning* yang mampu mengklasifikasikan emosi pada teks *chat* secara efektif, sehingga dapat memberikan kontribusi dalam pengembangan aplikasi yang lebih interaktif dan adaptif.

3.4.2 Studi literatur

Pada tahapan studi literatur, dilakukan pengumpulan berbagai teori dan penelitian terdahulu yang berkaitan dengan klasifikasi emosi berbasis teks. Studi ini mencakup

pemahaman tentang konsep emosi sebagai salah satu aspek penting dalam komunikasi manusia, serta bagaimana emosi dapat direpresentasikan melalui teks. Literatur yang dikaji juga membahas teknik NLP yang digunakan untuk mengolah teks, mulai dari proses tokenisasi, pembersihan data, hingga representasi kata dalam bentuk vektor numerik. Studi literatur juga membahas perkembangan metode *machine learning* dan *deep learning* yang sering digunakan dalam penelitian analisis teks, khususnya metode RNN.

3.4.3 Pengumpulan data

Tahap pengumpulan data merupakan proses awal dalam penelitian ini. Data yang digunakan berasal dari dataset teks yang berisi berbagai kalimat singkat dengan label emosi tertentu. Label emosi yang digunakan mencakup empat kategori utama, yaitu *joy*, *sadness*, *anger*, dan *fear*. Dataset ini dipilih karena relevan dengan tujuan penelitian, yakni mengklasifikasikan emosi dalam interaksi berbasis teks. Pada tahap ini, data dikumpulkan, diperiksa kelengkapan dan kualitasnya, serta dipersiapkan untuk tahap *preprocessing*.

Sumber: <https://www.kaggle.com/datasets/praveengovi/emotions-dataset-for-nlp>
(Praveen, 2020)

3.4.4 Preprocessing dan representasi data

Sebelum data teks dapat digunakan untuk melatih model, data tersebut perlu melalui tahap *preprocessing*. Tahap ini bertujuan untuk membersihkan dan

menstandarisasi teks agar lebih mudah diproses oleh model. Proses yang dilakukan antara lain:

1. *Text Cleaning*

Text cleaning merupakan tahap awal yang dilakukan untuk membersihkan teks dari karakter yang tidak relevan, seperti angka, tanda baca berlebihan, atau simbol yang tidak berhubungan dengan makna kalimat. Dengan membersihkan teks, model dapat lebih fokus pada isi kalimat tanpa terganggu oleh elemen tambahan.

Tabel 3.1. Proses *text cleaning*

Sebelum	Sesudah
I'm sooo angry!!! ☹️☹️ Why did he lie???	I am so angry Why did he lie

2. *Tokenization*

Tokenization adalah proses memecah kalimat menjadi potongan-potongan kata (token). Hal ini penting agar setiap kata dapat diproses secara individual oleh model. *Tokenization* juga membantu model memahami urutan kata dalam kalimat.

Tabel 3.2 Proses *tokenization*

Sebelum	Sesudah
I feel verysad today	['I', 'feel', 'very', 'sad', 'today']

3. *Case Folding*

Case folding dilakukan untuk menyeragamkan seluruh karakter huruf pada data teks dengan mengubah semua huruf menjadi huruf kecil (*lowercase*), dengan tujuan untuk menghilangkan perbedaan penulisan kata yang hanya disebabkan oleh penggunaan huruf kapital, sehingga kata yang memiliki makna sama dapat direpresentasikan secara konsisten sebelum diproses pada tahap berikutnya.

Tabel 3.3 Proses *case folding*

Sebelum	Sesudah
I Feel VerySad Today	i feel very sad today

4. *Stopword Removal*

Stopword removal dilakukan dengan menghapus kata-kata umum yang tidak memberikan makna penting, misalnya “dan”, “yang”, atau “di”. Penghapusan *stopword* membantu mengurangi kebisingan data dan membuat model lebih fokus pada kata-kata yang benar-benar menentukan emosi.

Tabel 3.4. Proses *stopword removal*

Sebelum	Sesudah
I amafraid in the dark room	afraid dark room

5. *Stemming*

Stemming digunakan untuk mengembalikan kata ke bentuk dasarnya. Proses ini membantu menyatukan berbagai variasi kata sehingga model tidak perlu menghafal bentuk kata yang berbeda untuk makna yang sama.

Tabel 3.5. Proses *stemming*

Sebelum	Sesudah
She was crying because she lost her friend	cri lost friend

6. *Label Encoding*

Label encoding dilakukan untuk mengubah data kategorikal menjadi representasi numerik.

Tabel 3.6. Proses *label encoding*

Sebelum	Sesudah
anger, fear, joy, sadness	0, 1, 2, 3

7. *Padding Sequence*

Padding sequence adalah proses menyamakan panjang kalimat dengan menambahkan elemen kosong (padding) pada teks yang lebih pendek. Tujuannya agar semua *input* memiliki ukuran yang seragam sehingga dapat diproses oleh model dengan baik.

Tabel 3.7. Proses *Padding sequence*

Sebelum	Sesudah
["I", "angry"]	["I", "angry", "<PAD>", "<PAD>", "<PAD>"]

3.4.5 Pemodelan data

Pemodelan data dalam penelitian ini dilakukan menggunakan RNN, yang merupakan algoritma berbasis *deep learning* yang efektif dalam mengolah data sekuensial seperti teks. RNN mampu mempertahankan konteks urutan kata, sehingga sangat relevan digunakan untuk klasifikasi emosi. Arsitektur model dirancang dengan beberapa lapisan:

1. *Embedding layer*

Embedding layer merupakan lapisan *input* yang digunakan untuk mengubah kata dalam teks menjadi representasi numerik berbentuk vektor. Representasi ini membantu model memahami hubungan semantik antar kata, sehingga kata yang mirip maknanya memiliki vektor yang relatif berdekatan.

2. *Hidden layer*

Hidden layer adalah lapisan tengah dalam arsitektur *Recurrent Neural Network* (RNN) yang berfungsi memproses urutan vektor dari embedding. Pada lapisan ini, jaringan mempelajari pola dan konteks antar kata, termasuk urutan kemunculannya, sehingga model dapat menangkap makna yang lebih dalam dari sebuah kalimat.

3. *Output layer*

Output layer, yang biasanya berupa dense layer dengan fungsi aktivasi *softmax*, menghasilkan prediksi akhir dalam bentuk kelas emosi. Lapisan ini memetakan hasil pemrosesan hidden layer ke dalam kategori emosi yang sudah ditentukan, misalnya *joy*, *sadness*, *anger*, atau *fear*.

Tabel 3.8. Proses *embedding*, *hidden* dan *output layer*

<i>Input</i>	<i>Embedding</i>	<i>Hidden</i>	<i>Output</i>
I amsad today	[0.12, 0.53,]	(RNN membaca urutan <i>vector</i>)	Sadness

3.4.6 Pelatihan model

Tahap pelatihan dilakukan setelah model dirancang. Proses ini melibatkan pemberian data latih ke dalam model dengan hyperparameter tertentu, seperti jumlah *epoch* (50), ukuran *batch* (32), dan *learning rate* (0,0003). Selama pelatihan, model akan memperbarui bobotnya secara bertahap menggunakan algoritma BPTT agar mampu meminimalkan kesalahan prediksi. Hasil dari tahap ini adalah model terlatih yang siap digunakan untuk pengujian pada data uji.

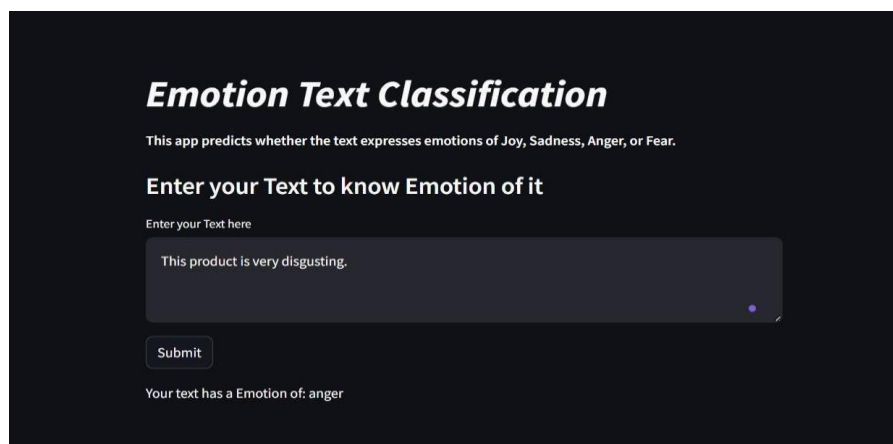
3.4.7 Evaluasi hasil

Evaluasi hasil dilakukan untuk menilai performa klasifikasi emosi pada teks *chat*. Ukuran yang digunakan meliputi akurasi, *precision*, *recall*, dan *F1-score*. Akurasi menunjukkan ketepatan prediksi secara keseluruhan, *precision* menekankan pada ketepatan hasil klasifikasi terhadap kelas tertentu, *recall* mengukur kemampuan model dalam menemukan data yang benar-benar termasuk dalam suatu kelas, dan *F1-score* menyeimbangkan *precision* dan *recall*. Selain itu, *confusion matrix* dapat

digunakan untuk menggambarkan distribusi hasil prediksi, sehingga dapat terlihat jelas apakah model mengklasifikasikan setiap emosi dengan benar atau masih terdapat kesalahan. Selain evaluasi performa model secara keseluruhan, penelitian ini juga melakukan analisis terhadap pengaruh panjang teks dalam proses klasifikasi emosi. Analisis dilakukan dengan mengelompokkan data berdasarkan panjang teks tertentu, kemudian membandingkan hasil klasifikasi pada masing-masing kelompok.

3.4.8 Visualisasi

Pada tahap visualisasi hasil, teks *chat* yang dimasukkan pengguna dianalisis menggunakan model yang telah dikembangkan. Hasil klasifikasi emosi tersebut kemudian ditampilkan melalui antarmuka berbasis web sederhana yang dibangun menggunakan pustaka Streamlit. Antarmuka ini memungkinkan pengguna untuk mengetikkan kalimat secara langsung, lalu sistem akan menampilkan prediksi emosi yang sesuai berdasarkan pemrosesan model.



Gambar 3.2. Visualisasi streamlit

Gambar 3.2 memperlihatkan tampilan antarmuka Streamlit untuk klasifikasi emosi dari teks. Antarmuka ini berfungsi untuk memprediksi emosi yang terkandung dalam suatu teks, dengan beberapa jenis emosi meliputi *joy*, *sadness*, *anger*, dan *fear*. Pengguna dapat memasukkan sebuah kalimat pada kotak *input* yang tersedia, kemudian menekan tombol *Submit* untuk memproses teks tersebut.

BAB IV

HASIL DAN PEMBAHASAN

4.1 PENGUMPULAN DATA

Dataset yang digunakan dalam penelitian ini merupakan dataset emosi berbasis teks yang diperoleh dari platform Kaggle dengan judul *Emotions Dataset for NLP*. Dataset tersebut berisi kumpulan kalimat atau teks pendek yang telah diberi label emosi sehingga dapat digunakan untuk membangun model klasifikasi emosi. Setiap data terdiri atas dua atribut utama, yaitu teks dan label emosi. Label emosi yang tersedia pada dataset mencakup empat kategori, yaitu *joy*, *sadness*, *anger*, dan *fear*. Dataset dipilih karena memiliki karakteristik yang sesuai dengan tujuan penelitian, yaitu melakukan klasifikasi emosi pada teks menggunakan metode RNN. Selain itu, data yang digunakan telah melalui proses pelabelan sehingga dapat langsung dimanfaatkan pada tahap pelatihan dan pengujian model. Sebelum digunakan dalam proses klasifikasi, dataset terlebih dahulu diperiksa untuk memastikan tidak terdapat data yang hilang (*missing value*) maupun data yang tidak sesuai dengan kebutuhan penelitian.

Tabel 4.1 Struktur dataset

Kolom	Jumlah Data	Tipe Data
Text	16000	object
Emotion	16000	object

Tabel 4.2 Hasil pemeriksaan missing value

Text	0
Emotion	0

Berdasarkan hasil pemeriksaan, tidak ditemukan data yang memiliki nilai kosong sehingga seluruh data dapat digunakan pada tahap penelitian berikutnya.

4.1.1 Distribusi dataset

Setelah proses seleksi kelas emosi dilakukan, dataset yang digunakan dalam penelitian ini terdiri atas empat kategori emosi, yaitu *joy*, *sadness*, *anger*, dan *fear*. Distribusi data pada masing-masing kategori emosi dianalisis untuk mengetahui keseimbangan jumlah data yang digunakan selama proses pelatihan dan pengujian model. Analisis distribusi data penting dilakukan karena jumlah data yang terlalu tidak seimbang dapat memengaruhi kemampuan model dalam mempelajari pola dari setiap kelas emosi. Oleh karena itu, distribusi dataset ditampilkan dalam bentuk tabel sehingga dapat memberikan gambaran mengenai komposisi data yang digunakan pada penelitian ini.

Tabel 4.3 Distribusi data pada setiap kelas emosi

Emosi	Jumlah Data
joy	5361
sadness	4666
anger	2159
fear	1937

Berdasarkan hasil distribusi data, setiap kategori emosi memiliki jumlah data yang berbeda. Meskipun demikian, keseluruhan data masih dapat digunakan untuk proses pembelajaran model karena seluruh kelas memiliki jumlah data yang memadai untuk dilakukan klasifikasi. Informasi distribusi data ini selanjutnya menjadi dasar dalam proses pembagian data ke dalam data pelatihan, data validasi, dan data pengujian.

4.2 PREPROCESSING DATA

Sebelum data masuk ke tahap pelabelan, dilakukan proses *preprocessing* terlebih dahulu untuk memastikan data dalam kondisi siap diproses oleh model. Tujuan utama dari *preprocessing* adalah membersihkan dan mentransformasikan data teks sehingga lebih mudah dipahami oleh model pembelajaran mesin. Tahapan *preprocessing* yang dilakukan meliputi *text cleaning*, *case folding*, *tokenization*, *stopword removal*, dan *stemming*.

Tabel 4.4 Proses *text cleaning*

Sebelum	Sesudah
i didnt feel humiliated...😊	i didnt feel humiliated

Text cleaning dilakukan untuk membersihkan teks dari karakter-karakter yang tidak diperlukan dalam proses klasifikasi. Karakter yang dihapus meliputi tanda baca, simbol, angka, maupun karakter khusus lainnya yang tidak memiliki kontribusi terhadap makna emosi dalam teks.

Tabel 4.5 Proses *case folding*

Sebelum	Sesudah
I DIDNT Feel Humiliated	i didnt feel humiliated

Setelah proses pembersihan teks selesai dilakukan, seluruh huruf pada teks diubah menjadi huruf kecil (*lowercase*). Tujuan dari *case folding* adalah menghindari perbedaan representasi kata yang sebenarnya memiliki makna yang sama. Sebagai contoh, kata “Didnt”, “DIDNT”, dan “didnt” akan dianggap sebagai satu kata yang sama setelah proses *case folding* dilakukan.

Tabel 4.6 Proses *tokenization*

Sebelum	Sesudah
i didnt feel humiliated	['i', 'didnt', 'feel', 'humiliated']

Tahap *tokenization* merupakan proses pemecahan teks menjadi unit-unit kata yang lebih kecil yang disebut token. Pada tahap ini, setiap kalimat diubah menjadi daftar kata sehingga dapat diproses lebih lanjut oleh sistem. Proses tokenisasi membantu model dalam mengenali struktur kalimat dan hubungan antar kata yang terdapat dalam teks.

Tabel 4.7 Proses *stopword removal*

Sebelum	Sesudah
i didnt feel humiliated	didnt feel humiliated

Tahap berikutnya adalah *stopword removal*. Pada tahap ini, kata-kata umum dalam bahasa Inggris yang tidak memberikan kontribusi signifikan terhadap proses klasifikasi dihapus dari teks. Beberapa contoh kata yang termasuk *stopword* adalah “the”, “is”, “and”, “of”, dan “to”.

Tabel 4.8 Proses *stemming*

Sebelum	Sesudah
didnt feel humiliated	didnt feel humili

Tahap *stemming* dilakukan untuk mengubah kata menjadi bentuk dasarnya. Kata-kata yang memiliki variasi bentuk akan disederhanakan menjadi satu bentuk dasar sehingga dapat mengurangi jumlah variasi kata yang dipelajari oleh model. Sebagai contoh, kata “*humiliate*”, “*humiliated*”, dan “*humilitaing*” dapat direduksi menjadi bentuk dasar yang sama, sehingga proses pembelajaran model menjadi lebih efisien karena kata-kata yang memiliki makna serupa direpresentasikan dalam bentuk yang lebih konsisten.

4.3 PEMBENTUKAN DAN REPRESENTASI DATA

Setelah proses *preprocessing* selesai, data teks yang telah bersih perlu diubah ke dalam bentuk representasi numerik agar dapat diproses oleh RNN. Pada tahap ini dilakukan beberapa proses, termasuk *label encoding* untuk mengubah label emosi menjadi nilai numerik, pembentukan *tokenizer* dan *vocabulary* untuk merepresentasikan kata dalam bentuk indeks, dan *padding sequence* untuk menyeragamkan panjang data masukan.

Tabel 4.9 Hasil *label encoding*

Emosi	Label Encoding
anger	0
fear	1
joy	2
sadness	3

Tabel 4.10 Contoh hasil *tokenizer*

Teks Hasil Preprocessing	Hasil Tokenizer
didnt feel humili	[8, 3, 524]

Tabel 4.11 Contoh hasil *padding sequence*

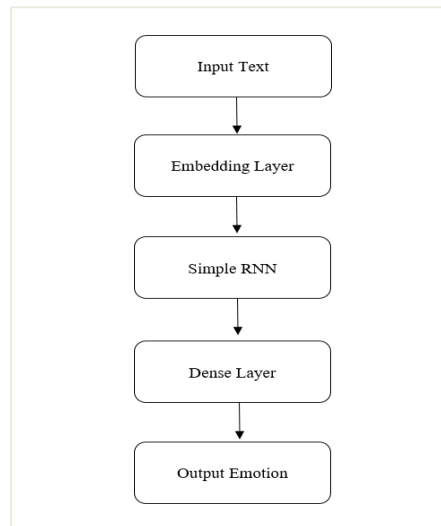
Sebelum	Sesudah
[8, 3, 524]	[0, 0, 0,....., 8, 3, 524]

4.4 IMPLEMENTASI MODEL

Setelah data berhasil direpresentasikan ke dalam bentuk numerik, selanjutnya dibangun dan dilatih model untuk mempelajari hubungan antar kata dalam teks sehingga dapat mengenali pola emosi yang terkandung di dalamnya. Implementasi model mencakup perancangan arsitektur jaringan, penentuan hyperparameter, serta proses pelatihan model menggunakan data yang telah dipersiapkan pada tahap sebelumnya.

4.4.1 Arsitektur model

Model dibangun menggunakan beberapa lapisan yang saling terhubung, yaitu *Embedding Layer*, *Simple RNN Layer*, dan *Dense Layer*. Setiap lapisan memiliki fungsi yang berbeda dalam proses pembelajaran, mulai dari representasi kata, pemrosesan urutan teks, hingga menghasilkan prediksi emosi.



Gambar 4.1 Arsitektur model

Tabel 4.12 Struktur arsitektur model

Layer	Output Shape	Parameter	Keterangan
Embedding	0	1513050	Representasi kata menjadi vektor
Simple RNN	1	5856	Mempelajari urutan kata
Dense	2	132	Prediksi kelas emosi

Berdasarkan Tabel 4.12, arsitektur model yang digunakan pada penelitian ini terdiri atas tiga lapisan utama, yaitu *Embedding*, *Simple RNN*, dan *Dense*. Lapisan *Embedding* memiliki parameter terbanyak karena bertugas membentuk representasi vektor dari setiap kata dalam *vocabulary*. Selanjutnya, lapisan *Simple RNN* mempelajari hubungan antar kata berdasarkan urutan kemunculannya dalam teks, sedangkan lapisan *Dense* menghasilkan prediksi akhir terhadap empat kategori emosi yang digunakan pada penelitian ini.

4.4.2 Konfigurasi *hyperparameter*

Sebelum proses pelatihan dilakukan, beberapa *hyperparameter* ditentukan untuk mengatur cara model belajar dari data. *Hyperparameter* yang digunakan meliputi *learning rate*, jumlah unit pada lapisan RNN, dimensi *embedding*, jumlah *epoch*, serta parameter lain yang mendukung proses pelatihan model.

Tabel 4.13 *Hyperparameter* model

Hyperparameter	Nilai
Learning Rate	0,0003
Optimizer	Adam
Embedding Dimension	150
RNN Units	32
Max Epoch	50
Patience	5
Max Sequence Length	35
Activation Output	Softmax

Model dikonfigurasi menggunakan beberapa *hyperparameter* yang telah ditentukan sebelum proses pelatihan. *Learning rate* sebesar 0,0003 digunakan untuk mengatur laju pembaruan bobot selama proses pembelajaran, *optimizer* Adam dipilih karena mampu memberikan proses optimasi yang stabil. Dimensi *embedding* sebesar 150 dan *Simple* RNN dengan 32 unit digunakan untuk merepresentasikan kata serta mempelajari hubungan antar kata dalam teks. Model dilatih hingga maksimum 50 *epoch* dengan mekanisme *Early Stopping* yang memiliki nilai *patience* sebesar 5, menggunakan panjang sekuens maksimum berupa 35 kata, serta fungsi aktivasi *Softmax* pada lapisan keluaran untuk menghasilkan probabilitas dari setiap kelas emosi.

4.4.3 Proses pelatihan model

Setelah arsitektur dan hyperparameter ditentukan, model kemudian dilatih menggunakan data pelatihan yang telah dipersiapkan pada tahap sebelumnya. Selama proses pelatihan, model mempelajari pola hubungan antara teks dan label emosi melalui beberapa iterasi (*epoch*). Untuk memantau performa model, digunakan data validasi dan metrik akurasi. Mekanisme *Early Stopping* digunakan untuk menghentikan proses pelatihan secara otomatis apabila tidak terjadi peningkatan performa dalam sejumlah epoch tertentu.

```
Epoch 1/50
442/442 [=====] - 16s 32ms/step - loss: 1.2597 - accuracy: 0.4420 - val_loss: 1.1273 - val_accuracy: 0.5594
Epoch 2/50
442/442 [=====] - 13s 30ms/step - loss: 0.7439 - accuracy: 0.7423 - val_loss: 0.5670 - val_accuracy: 0.7949
Epoch 3/50
442/442 [=====] - 13s 30ms/step - loss: 0.3381 - accuracy: 0.9060 - val_loss: 0.4268 - val_accuracy: 0.8610
Epoch 4/50
442/442 [=====] - 13s 29ms/step - loss: 0.1943 - accuracy: 0.9520 - val_loss: 0.3828 - val_accuracy: 0.8788
Epoch 5/50
442/442 [=====] - 13s 30ms/step - loss: 0.1266 - accuracy: 0.9686 - val_loss: 0.3696 - val_accuracy: 0.8794
Epoch 6/50
442/442 [=====] - 13s 30ms/step - loss: 0.0869 - accuracy: 0.9792 - val_loss: 0.3898 - val_accuracy: 0.8719
Epoch 7/50
442/442 [=====] - 13s 30ms/step - loss: 0.0676 - accuracy: 0.9837 - val_loss: 0.4874 - val_accuracy: 0.8627
Epoch 8/50
442/442 [=====] - 14s 31ms/step - loss: 0.0498 - accuracy: 0.9901 - val_loss: 0.3959 - val_accuracy: 0.8736
Epoch 9/50
442/442 [=====] - 13s 30ms/step - loss: 0.0380 - accuracy: 0.9916 - val_loss: 0.3988 - val_accuracy: 0.8759
Epoch 10/50
442/442 [=====] - 13s 31ms/step - loss: 0.0284 - accuracy: 0.9943 - val_loss: 0.4128 - val_accuracy: 0.8800
Epoch 11/50
442/442 [=====] - 13s 30ms/step - loss: 0.0276 - accuracy: 0.9943 - val_loss: 0.4594 - val_accuracy: 0.8616
Epoch 12/50
442/442 [=====] - 13s 30ms/step - loss: 0.0203 - accuracy: 0.9957 - val_loss: 0.4612 - val_accuracy: 0.8679
Epoch 13/50
...
Epoch 14/50
442/442 [=====] - 13s 30ms/step - loss: 0.0129 - accuracy: 0.9970 - val_loss: 0.4855 - val_accuracy: 0.8696
Epoch 15/50
442/442 [=====] - 14s 31ms/step - loss: 0.0112 - accuracy: 0.9975 - val_loss: 0.5004 - val_accuracy: 0.8702
Output is truncated. View as a scrollable element or open in a text editor. Adjust cell output settings...
```

Gambar 4.2 Proses pelatihan model

Tabel 4.14 Ringkasan hasil pelatihan

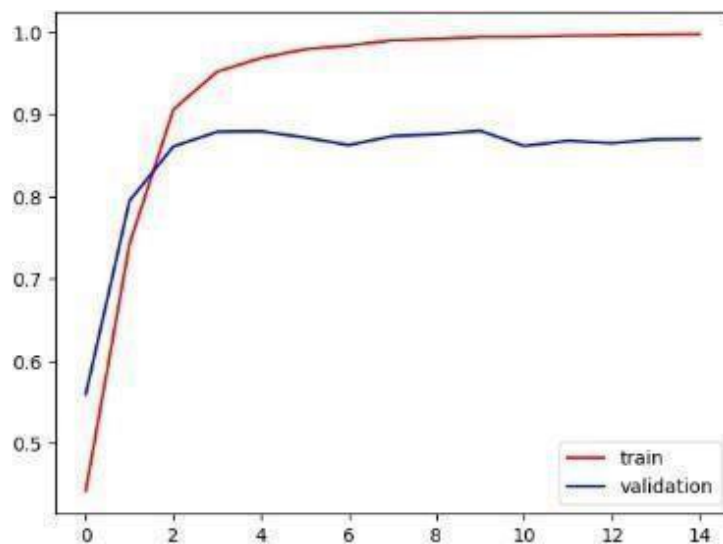
Parameter	Nilai
Epoch Maksimum	50
Epoch Berhenti	15
Early Stopping	Yes
Restore Best Weights	Yes

Proses pelatihan model berhenti pada epoch ke-15 meskipun batas maksimum pelatihan ditetapkan sebanyak 50 epoch. Hal ini menunjukkan bahwa mekanisme *Early Stopping* berhasil menghentikan pelatihan ketika performa model tidak lagi

mengalami peningkatan dan secara otomatis mengembalikan bobot terbaik yang diperoleh selama proses pelatihan.

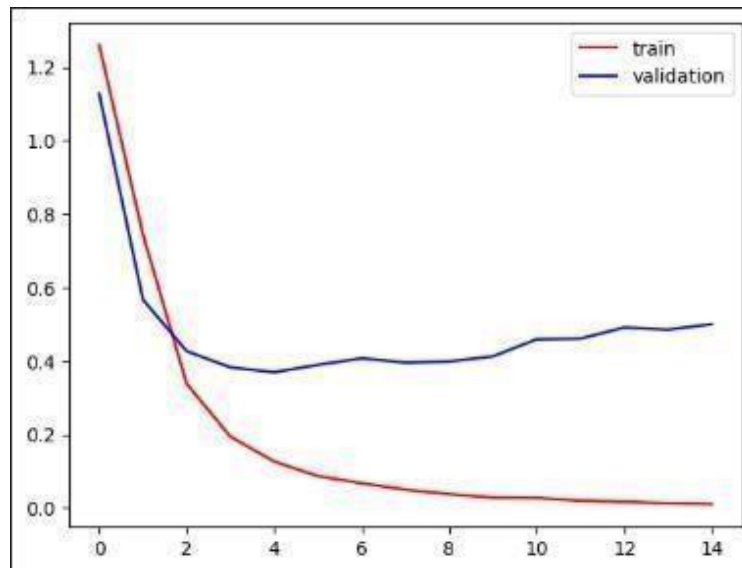
4.5 HASIL PELATIHAN MODEL

Tahap berikutnya adalah menganalisis performa model selama proses pembelajaran. Analisis dilakukan dengan mengamati perubahan nilai akurasi dan *loss* pada data pelatihan maupun data validasi selama setiap *epoch*. Hasil pelatihan ini digunakan untuk mengetahui kemampuan model dalam mempelajari pola emosi pada data teks serta mengevaluasi apakah model mengalami *underfitting*, *overfitting*, atau telah mencapai performa yang optimal.



Gambar 4.3 Grafik *accuracy* model

Nilai *accuracy* digunakan untuk mengukur tingkat ketepatan model dalam mengklasifikasikan emosi selama proses pelatihan. Grafik *training accuracy* dan *validation accuracy* menunjukkan perkembangan performa model pada setiap *epoch*. Semakin tinggi nilai *accuracy*, semakin baik kemampuan model dalam mengenali pola emosi yang terdapat pada dataset.



Gambar 4.4 Grafik *loss* model

Selain *accuracy*, proses pelatihan juga dievaluasi menggunakan nilai *loss*. Nilai *loss* menunjukkan tingkat kesalahan model dalam melakukan prediksi. Semakin kecil nilai *loss*, semakin baik kemampuan model dalam mempelajari pola dari data yang digunakan. Grafik *training loss* dan *validation loss* digunakan untuk melihat kestabilan proses pembelajaran model selama pelatihan berlangsung.

Tabel 4.15 Hasil pelatihan model

Parameter	Nilai
Learning Rate	0,0003
Epoch Maksimum	50
Epoch Berhenti	15
Optimizer	Adam
Early Stopping	Yes
Restore Best Weights	Yes

Akurasi Terbaik	0,9975
Validation Accuracy Terbaik	0,8702

Berdasarkan Tabel 4.15, model berhasil menyelesaikan proses pelatihan pada epoch ke- 15 dari maksimum 50 epoch karena mekanisme *early stopping* menghentikan pelatihan ketika performa validasi tidak lagi mengalami peningkatan. Selama proses pelatihan, model memperoleh akurasi pelatihan terbaik sebesar **99,75%** dan akurasi validasi terbaik sebesar **87,02%**, sehingga model yang memiliki performa terbaik digunakan pada tahap pengujian.

4.6 EVALUASI MODEL

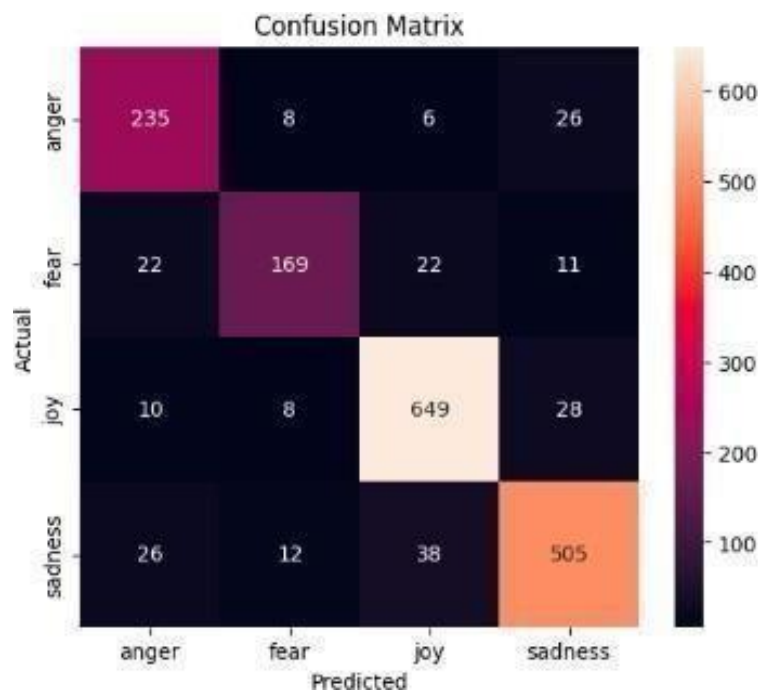
Selanjutnya, model dievaluasi menggunakan data pengujian untuk mengetahui kemampuan model dalam mengklasifikasikan emosi pada data yang belum pernah dilihat sebelumnya. Evaluasi dilakukan menggunakan *classification report* dan *confusion matrix*. Kedua metode evaluasi tersebut digunakan untuk memberikan gambaran yang lebih rinci mengenai performa model, baik secara keseluruhan maupun pada masing-masing kategori emosi.

Tabel 4.16 Hasil *classification report*

	precision	recall	f1-score	support
anger	0.80	0.85	0.83	275
fear	0.86	0.75	0.80	224
joy	0.91	0.93	0.92	695
sadness	0.89	0.87	0.88	581
accuracy	-	-	0.88	1775

Berdasarkan hasil *classification report*, model memperoleh akurasi keseluruhan sebesar 88% pada data pengujian. Kelas *joy* memiliki performa terbaik

dengan nilai *f1-score* sebesar 0,92, sedangkan kelas *fear* memiliki nilai *f1-score* terendah sebesar 0,80. Secara keseluruhan, nilai *precision*, *recall*, dan *f1-score* pada setiap kelas menunjukkan bahwa model mampu mengklasifikasikan emosi dengan baik dan memberikan performa yang konsisten pada seluruh kategori emosi yang digunakan.



Gambar 4.5 *Confusion matrix*

Berdasarkan gambar 4.5, sebagian besar data berhasil diklasifikasikan dengan benar, yang ditunjukkan oleh tingginya nilai pada diagonal utama confusion matrix. Model mengenali kelas *joy* dengan sangat baik, ditunjukkan oleh 649 data yang berhasil diprediksi dengan benar dari total 695 data. Kelas *sadness* juga menunjukkan performa yang baik dengan 505 prediksi benar dari 581 data, diikuti oleh kelas *anger* sebanyak 235 prediksi benar dari 275 data dan kelas *fear* sebanyak 169 prediksi benar dari 224 data. Meskipun demikian, masih terdapat beberapa kesalahan klasifikasi, terutama antara kelas *sadness* dan *joy*, serta antara *anger* dan *sadness*. Secara keseluruhan, hasil confusion matrix menunjukkan bahwa RNN mampu membedakan keempat kategori emosi dengan baik, sejalan dengan akurasi pengujian yang mencapai 88%.

4.7 ANALISIS PENGARUH PANJANG TEKS

Analisis ini dilakukan untuk mengetahui apakah jumlah kata dalam suatu teks memengaruhi tingkat akurasi prediksi yang dihasilkan model. Untuk keperluan analisis, data pengujian dikelompokkan ke dalam beberapa kategori panjang teks berdasarkan jumlah kata yang terkandung pada setiap kalimat, kemudian tingkat akurasi pada masing-masing kategori dibandingkan. eks dengan jumlah kata kurang dari atau sama dengan 10 dikategorikan sebagai *short*, teks dengan jumlah kata 11 hingga 26 dikategorikan sebagai *medium*, sedangkan teks dengan jumlah kata lebih dari 26 dikategorikan sebagai *long*.

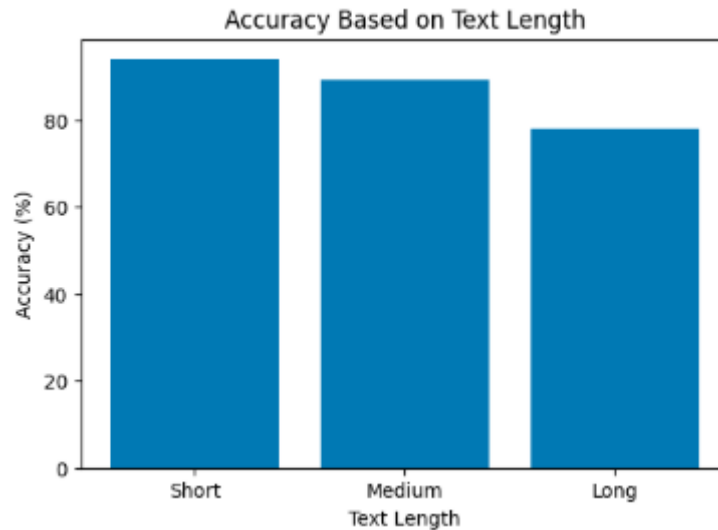
Tabel 4.17 Distribusi data berdasarkan panjang teks

Panjang Kata	Jumlah Data
Short	454
Medium	918
Long	403

Berdasarkan Tabel 4.17, kategori *medium* memiliki jumlah data paling banyak dibandingkan kategori lainnya. Sementara itu, kategori *short* dan *long* memiliki jumlah data yang lebih sedikit, namun tetap memiliki jumlah sampel yang memadai untuk dilakukan analisis performa model.

Tabel 4.18 Akurasi data berdasarkan panjang teks

Panjang Kata	Jumlah Data	Akurasi (%)
Short	454	93.83
Medium	918	89.11
Long	403	77.92



Gambar 4.6 Perbandingan akurasi berdasarkan panjang teks

Berdasarkan Tabel 4.18 dan Gambar 4.6, model memperoleh tingkat akurasi tertinggi pada kategori teks pendek dengan nilai sebesar 93,83%. Akurasi kemudian menurun pada kategori teks sedang menjadi 89,11% dan kembali menurun pada kategori teks panjang menjadi 77,92%. Hasil tersebut menunjukkan bahwa semakin panjang teks yang diproses, tingkat akurasi model cenderung mengalami penurunan.

Perbedaan tingkat akurasi pada setiap kategori panjang teks menunjukkan bahwa panjang teks dapat mempengaruhi kemampuan model dalam mengenali emosi. Teks yang lebih pendek umumnya mengandung informasi yang lebih terfokus sehingga emosi yang disampaikan lebih mudah dikenali oleh model. Sebaliknya, teks yang lebih panjang cenderung memiliki variasi kata dan konteks yang lebih banyak, sehingga proses identifikasi emosi menjadi lebih kompleks.

Hasil analisis ini menunjukkan bahwa model yang digunakan mampu memberikan performa yang sangat baik pada teks pendek dan cukup baik pada teks sedang. Namun demikian, performa model mengalami penurunan pada teks yang lebih panjang.

4.8 ANALISIS PENGARUH *KEYWORD*

Selain dipengaruhi oleh panjang teks, hasil klasifikasi emosi juga dapat dipengaruhi oleh keberadaan kata-kata tertentu yang memiliki keterkaitan kuat dengan suatu emosi. Kata-kata tersebut berpotensi menjadi fitur penting yang

digunakan model dalam menentukan kategori emosi pada sebuah teks.

4.8.1 *Keyword* emosi

Pengujian dilakukan dengan memasukkan beberapa kalimat yang mengandung kata kunci tertentu ke dalam sistem. Kata kunci yang digunakan dipilih berdasarkan keterkaitannya dengan kategori emosi yang terdapat pada penelitian ini, yaitu *joy*, *sadness*, *anger*, dan *fear*. Contoh *keyword* dapat dilihat di tabel 4.19 dibawah ini.

Tabel 4.19 *Keyword* emosi

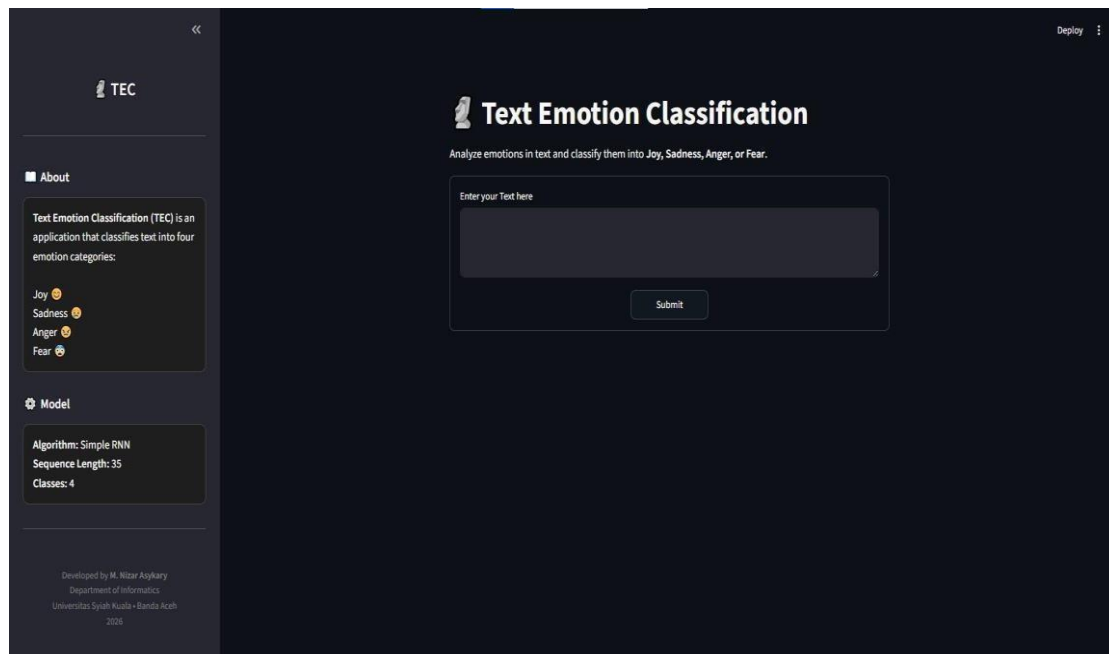
Emosi	Keyword
Joy	happy, excited, wonderful, amazing, grateful
Sadness	sad, lonely, depressed, disappointed, hopeless
Anger	angry, hate, annoyed, mad
Fear	afraid, scared, terrified

Berdasarkan hasil pengujian, didapatkan data bahwa keberadaan kata-kata yang memiliki muatan emosional kuat memberikan pengaruh terhadap keputusan klasifikasi yang dihasilkan model. Hal ini menunjukkan bahwa model mampu mengenali pola kata yang sering muncul pada masing-masing kategori emosi selama proses pelatihan. Meskipun demikian, hasil prediksi tidak hanya ditentukan oleh satu kata tertentu, tetapi juga dipengaruhi oleh konteks kalimat secara keseluruhan. Kombinasi kata dan susunan kalimat tetap berperan penting dalam menentukan emosi yang diprediksi oleh model.

4.9 IMPLEMENTASI SISTEM

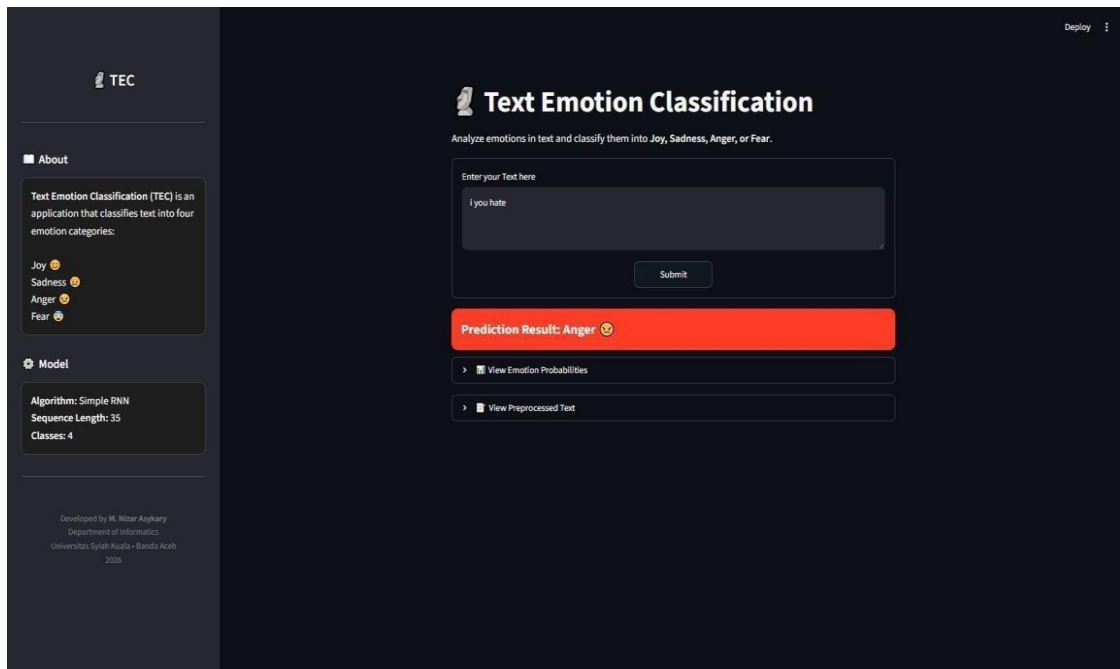
Tahap terakhir adalah mengimplementasikan model ke dalam sebuah sistem berbasis web menggunakan Streamlit. Implementasi model dilakukan agar pengguna dapat melakukan klasifikasi emosi secara langsung melalui antarmuka yang sederhana dan mudah digunakan. Model yang dibangun memungkinkan pengguna

memasukkan teks, menjalankan proses klasifikasi, serta melihat hasil prediksi emosi yang dihasilkan oleh model.



Gambar 4.7 Halaman utama web

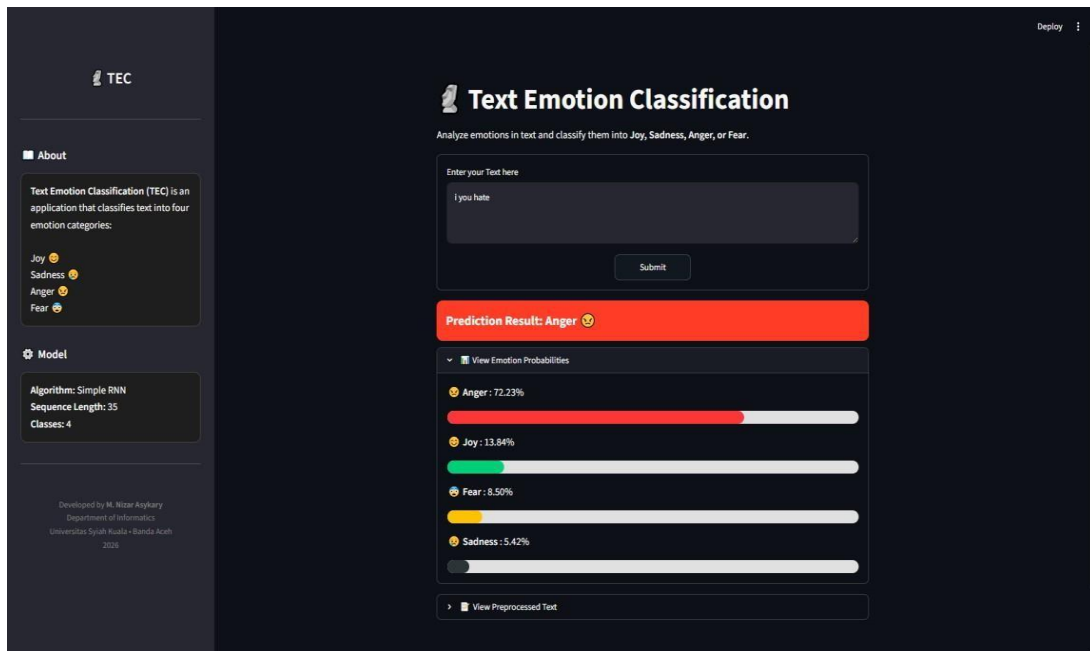
Halaman utama merupakan tampilan awal yang digunakan pengguna untuk berinteraksi dengan sistem. Pada halaman ini tersedia *text area* yang digunakan untuk memasukkan kalimat atau teks yang akan dianalisis. Selain itu, sistem juga menampilkan informasi singkat mengenai aplikasi dan model yang digunakan dalam proses klasifikasi emosi. Berdasarkan Gambar 4.7, sistem menyediakan antarmuka yang sederhana sehingga pengguna dapat langsung memasukkan teks yang ingin dianalisis tanpa memerlukan konfigurasi tambahan.



Gambar 4.8 Hasil prediksi emosi

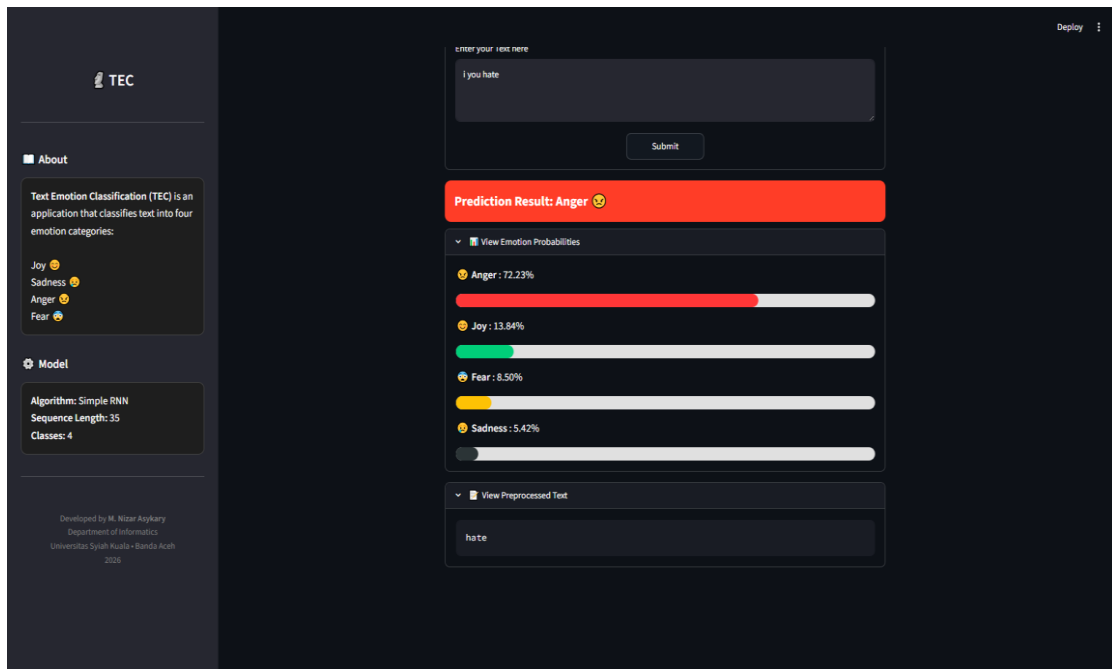
Setelah pengguna memasukkan teks dan menekan tombol “Submit”, sistem akan menampilkan hasil prediksi emosi yang diperoleh dari model. Hasil prediksi ditampilkan dalam bentuk kategori emosi beserta ikon yang merepresentasikan emosi tersebut. Tampilan ini dirancang agar hasil klasifikasi dapat dipahami dengan mudah oleh pengguna.

Selain digunakan untuk menguji kalimat yang terstruktur, implementasi sistem juga digunakan untuk mengamati kemampuan model dalam mengklasifikasikan emosi pada kalimat yang tidak terstruktur. Berdasarkan gambar 4.8 diatas, penulis sengaja memasukkan kalimat “*i you hate*”, yang seharusnya adalah “*i hate you*”, tetapi model masih mampu mengenali emosi dengan benar meskipun susunan kata tidak mengikuti struktur kalimat yang baik, dan tetap diklasifikasikan sebagai emosi *anger*. Pengujian pada aspek lain yang telah dilakukan adalah penggunaan kata singkatan, seperti kata “*u*” yang diambil dari kata “*you*” pada kalimat “*i hate u*”, juga masih dapat diproses dengan baik sehingga menghasilkan prediksi emosi yang sesuai. Hasil tersebut menunjukkan bahwa model memiliki kemampuan untuk mengenali pola kata-kata yang berkaitan dengan emosi meskipun bentuk kalimat tidak sepenuhnya terstruktur.



Gambar 4.9 Visualisasi probabilitas emosi

Selain menampilkan hasil prediksi utama, sistem juga menyediakan informasi mengenai probabilitas untuk setiap kategori emosi. Informasi ini ditampilkan dalam bentuk persentase dan grafik batang sehingga pengguna dapat melihat tingkat keyakinan model terhadap masing-masing kelas emosi. Dengan adanya visualisasi probabilitas, hasil klasifikasi menjadi lebih informatif dibandingkan hanya menampilkan satu kategori emosi saja. Berdasarkan Gambar 4.9, sistem menampilkan probabilitas dengan warna kustom yang berbeda untuk setiap emosi yang tersedia. Nilai probabilitas terbesar menunjukkan emosi yang dipilih sebagai hasil prediksi utama oleh model.



Gambar 4.10 Hasil *preprocessing* teks

Sistem juga menyediakan fitur untuk menampilkan hasil preprocessing terhadap teks yang dimasukkan pengguna. Fitur ini digunakan untuk memperlihatkan perubahan teks setelah melalui tahapan pembersihan data sebelum diproses oleh model. Dengan adanya fitur ini, pengguna dapat memahami bagaimana teks direpresentasikan sebelum dilakukan proses klasifikasi. Berdasarkan Gambar 4.10, sistem menampilkan hasil preprocessing yang digunakan sebagai masukan model. Teks yang ditampilkan telah melalui proses *text cleaning*, *case folding*, *stopword removal*, dan *stemming* sesuai tahapan yang diterapkan pada penelitian ini.

BAB V

KESIMPULAN DAN SARAN

5.1 KESIMPULAN

Berdasarkan hasil pengujian dan analisis yang telah dilakukan terhadap model RNN untuk klasifikasi emosi pada teks *chat*, maka dapat diambil beberapa kesimpulan sebagai berikut:

1. Penelitian ini berhasil mengimplementasikan pendekatan *Recurrent Neural Network* untuk melakukan klasifikasi emosi pada teks *chat* dengan melalui tahapan *preprocessing*, pembentukan representasi data, pelatihan model, evaluasi, hingga implementasi sistem berbasis Streamlit. Model yang dibangun mampu mengklasifikasikan empat kategori emosi, yaitu *anger*, *fear*, *joy*, dan *sadness*, sehingga tujuan utama penelitian untuk mengimplementasikan pendekatan RNN pada klasifikasi emosi berhasil dicapai.
2. Berdasarkan hasil evaluasi menggunakan *classification report* dan *confusion matrix*, model memperoleh tingkat akurasi sebesar 88% pada data pengujian. Nilai *precision*, *recall*, dan *f1-score* pada setiap kategori emosi menunjukkan bahwa model memiliki kemampuan yang baik dalam mengenali pola emosi pada teks. Selain itu, berdasarkan pengujian melalui implementasi sistem, model juga mampu mengklasifikasikan beberapa kalimat yang tidak terstruktur, seperti perubahan urutan kata maupun penggunaan singkatan dalam percakapan, sehingga menunjukkan bahwa model masih mampu mengenali pola kata yang merepresentasikan emosi meskipun bentuk kalimat tidak sepenuhnya mengikuti struktur bahasa yang baik.
3. Hasil analisis terhadap panjang teks menunjukkan bahwa panjang kalimat memberikan pengaruh terhadap performa klasifikasi emosi. Berdasarkan pengelompokan panjang teks menjadi kategori *short*, *medium*, dan *long*, diperoleh akurasi masing-masing sebesar 94,49%, 89,76%, dan 78,66%. Hasil tersebut menunjukkan bahwa model memberikan performa terbaik pada teks yang

lebih pendek, sedangkan akurasi cenderung menurun pada teks yang lebih panjang karena informasi yang diproses menjadi lebih kompleks dan berpotensi mengandung lebih banyak kata yang kurang relevan terhadap emosi utama.

4. Penggunaan tahapan *preprocessing* yang terdiri atas *text cleaning*, *case folding*, *tokenization*, *stopword removal*, *stemming*, dan representasi data menggunakan *label encoding*, *padding sequence* dan *Embedding Layer* memberikan masukan data yang lebih terstruktur bagi model. Tahapan tersebut membantu model dalam mempelajari hubungan antar kata secara lebih efektif sehingga menghasilkan performa klasifikasi yang baik selama proses pelatihan maupun pengujian.
5. Implementasi sistem menggunakan Streamlit berhasil menyediakan antarmuka yang sederhana dan mudah digunakan untuk melakukan klasifikasi emosi secara langsung. Sistem mampu menerima masukan berupa teks, melakukan seluruh tahapan pemrosesan secara otomatis, dan menampilkan hasil prediksi emosi kepada pengguna, sehingga model yang telah dibangun dapat diterapkan dalam bentuk aplikasi yang siap digunakan sebagai media demonstrasi maupun pengembangan lebih lanjut.

5.2 SARAN

Berdasarkan kesimpulan yang telah dijelaskan sebelumnya, maka penulis memberikan saran untuk penelitian selanjutnya sebagai berikut:

1. Penelitian selanjutnya dapat menggunakan arsitektur *deep learning* yang lebih kompleks, seperti LSTM ataupun *Transformer*, sehingga dapat dibandingkan performanya dengan model RNN dalam melakukan klasifikasi emosi pada teks.
2. Dataset yang digunakan dapat diperluas dengan jumlah data yang lebih banyak, variasi kosakata yang lebih beragam, mencakup berbagai bentuk bahasa percakapan, serta distribusi panjang teks yang lebih seimbang agar analisis pengaruh panjang teks terhadap akurasi model dapat dilakukan secara lebih optimal dan model mampu melakukan generalisasi dengan lebih baik terhadap

berbagai jenis teks.

3. Sistem yang telah dikembangkan dapat ditingkatkan dengan menambahkan fitur pendukung, seperti penyajian tingkat kepercayaan (*confidence score*) pada setiap hasil prediksi serta penyempurnaan antarmuka pengguna agar aplikasi menjadi lebih informatif, menarik, dan mudah digunakan.

DAFTAR PUSTAKA

- Amazon Web Services. (2024). *What is RNN?*
- Brownlee, J. (2021). *Deep Learning for Natural Language Processing: Develop Deep Learning Models for Your Natural Language Problems.*
- Chang, C. L., & Hsing, H. T. (2021). Emotion recognition and response generation in conversational agents using RNN-CNN hybrid networks with valence-arousal modeling. *Journal of Intelligent & Fuzzy Systems*, 40(5), 8911–8924.
- Chung, J., Gulcehre, C., Cho, K., & Bengio, Y. (2014). *Empirical Evaluation of Gated Recurrent Neural Networks on Sequence Modeling.* arXiv preprint arXiv:1412.3555.
- Chutia, S., Rajkonwar, M., & Sarmah, R. (2024). Enhancing emotion recognition in informal text using contextual deep learning approaches. *Neural Processing Letters*, 56(2), 2345–2362.
- Cho, K., van Merriënboer, B., Gulcehre, C., Bahdanau, D., Bougares, F., Schwenk, H., & Bengio, Y. (2014). *Learning Phrase Representations Using RNN Encoder–Decoder for Statistical Machine Translation.* arXiv preprint arXiv:1406.1078.
- DataCamp. (2023). *A Beginner's Guide to Recurrent Neural Networks (RNNs).*
- Demszky, D., Movshovitz-Attias, D., Ko, J., Cowen, A., Nemade, G., & Ravi, S. 2020. *GoEmotions: A dataset of fine-grained emotions.* *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, 4040–4054.
- Glenn, A., LaCasse, P., & Cox, B. (2023). Emotion Classification of Indonesian Tweets Using Bidirectional LSTM. *Neural Computing and Applications*, 35, 9567–9578.
- Gu, S., Wang, F., Patel, N. P., Bourgeois, J. A., & Huang, J. H. (2019). *A Model for Basic Emotions Using Natural Language Processing.* *Frontiers in Psychology*, 10, 1066.
- Hasan, M., Rahman, S., & Alam, M. (2022). Comparative performance of GRU and LSTM networks in emotion classification using textual data. *Journal of Information and Telecommunication*, 6(4), 512–525.
- Hinojosa Lee, C. (2024). *Evaluation metrics for machine learning models: Accuracy, precision, recall, and F1-score explained.* Towards Data Science.

- Kaur, R., & Kaur, P. (2023). *Emotion Classification Using Recurrent Neural Networks on Text Data*. *International Journal of Advanced Computer Science and Applications*, 14(2), 112–119.
- Kowsari, K., Brown, D. E., Heidarysafa, M., Meimandi, K. J., Gerber, M. S., & Barnes, L. E. (2019). *Text Classification Algorithms: A Survey*. *Information*, 10(4), 150.
- Krouska, A., Troussas, C., & Virvou, M. 2017. *Comparative evaluation of algorithms for sentiment analysis over social networking services*. *Journal of Universal Computer Science*, 23(8), 755–768.
- Kusal, T., Dissanayake, P., & Liyanage, C. 2023. *Deep learning-based emotion recognition from text: A comprehensive survey*. *Expert Systems with Applications*, 222, 119798.
- Manning, C. D., Raghavan, P., & Schütze, H. 2008. *Introduction to Information Retrieval*. Cambridge: Cambridge University Press.
- McKinney, W. 2021. *Python for Data Analysis: Data Wrangling with Pandas, NumPy, and Jupyter*. Sebastopol: O'Reilly Media.
- Medium. (2024). *Confusion Matrix for Multiclass Classification*.
- Mienye, I. D., Swart, T. G., & Obaido, G. (2024). Recurrent Neural Networks: A Comprehensive Review of Architectures, Variants, and Applications. *Information*, 15(9), 517.
- Mikolov, T., Chen, K., Corrado, G., & Dean, J. (2013). *Efficient Estimation of Word Representations in Vector Space*. arXiv preprint arXiv:1301.3781.
- Munezero, M., Montero, C. S., Sutinen, E., & Pajunen, J. (2014). Are they different? Affect, feeling, emotion, sentiment, and opinion detection in text. *IEEE Transactions on Affective Computing*, 5(2), 101–111.
- Pargent, F., Pfisterer, F., Thomas, J., & Bischl, B. (2022). *Regularized Target Encoding Outperforms Traditional Methods in Supervised Machine Learning with High Cardinality Features*. *Computational Statistics*, 37(5), 2671–2692.
- Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., ... & Duchesnay, E. 2011. *Scikit-learn: Machine Learning in Python*. *Journal of Machine Learning Research*, 12, 2825–2830.
- Praveen, G. (2022). Emotions Dataset for Natural Language Processing. Kaggle. <https://www.kaggle.com/datasets/praveengovi/emotions-dataset-for-nlp>
- Polivoda, M. (2021). Backpropagation Through Time: Understanding the training process of recurrent neural networks. *Medium Publication on Machine*

Learning.

- Poria, S., Cambria, E., Bajpai, R., & Hussain, A. (2019). A review of affective computing: From unimodal analysis to multimodal fusion. *Information Fusion*, 37, 98–125.
- QuarkML. (2023). *Backpropagation Through Time Explained*. QuarkML Blog.
- Rahmi, N. A., Rudiman, R., Septiana, R., & Suhalim, C. T. (2024). *Sentiment Analysis of Public Opinion Using Text Preprocessing and Machine Learning*. *JSE Journal of Science and Engineering*, 1(1), 23–29.
- Rainio, P. (2024). *Understanding Accuracy and Recall in Machine Learning Classification*. *Towards Data Science*.
- Rianto, R., Mutiara, A. B., Wibowo, E. P., & Santosa, P. I. (2021). *Improving the Accuracy of Text Classification Using Stemming Method, a Case of Non-Formal Indonesian Conversation*. *Journal of Big Data*, 8(26).
- Sailunaz, K., Dhaliwal, M., Rokne, J., & Alhajj, R. 2018. *Emotion detection from text and speech: A survey*. *Social Network Analysis and Mining*, 8(1), 28.
- Sharma, N., Singh, R., & Kaur, P. (2022). Contextual emotion detection in text using deep neural networks. *Expert Systems with Applications*, 198, 116834.
- Zhang, Y., Wang, S., & Liu, X. (2018). Deep learning for sentiment analysis: A survey. *Wiley Interdisciplinary Reviews: Data Mining and Knowledge Discovery*, 8(4), e1253.
- Zhou, J., Lin, W., & Li, Y. (2023). *Emotion Recognition from Text Using Deep Recurrent Models*. *Information Processing & Management*, 60(5), 103333.

LAMPIRAN

Lampiran 1. Kode Program *Import Library* dan Pembacaan Dataset

```
1 import numpy as np
2 import pandas as pd
3
4 import os
5 for dirname, _, filenames in os.walk('Dataset'):
6     for filename in filenames:
7         print(os.path.join(dirname, filename))your code here
8
9 import matplotlib.pyplot as plt
10 import seaborn as sns
11 import re
12 import gensim
13 from gensim.models import Word2Vec, KeyedVectors
14
15 train_df = pd.read_csv("../Dataset/train.txt", sep=';', header=None, names=['Text', 'Emotion'])
16 test_df = pd.read_csv("../Dataset/test.txt", sep=';', header=None, names=['Text', 'Emotion'])
17 val_df = pd.read_csv("../Dataset/val.txt", sep=';', header=None, names=['Text', 'Emotion'])
```

Lampiran 2. Kode Program *Preprocessing* Data

```
1 import nltk
2 nltk.download('stopwords')
3
4 from nltk.corpus import stopwords
5 stop_words = stopwords.words('english')
6 stop_words
7
8 from nltk.stem import SnowballStemmer
9 snowballstemmer=SnowballStemmer('english')
10
11 corpus = []
12
13 for index, row in training_dataset.iterrows():
14     text = re.sub('[^a-zA-Z]', ' ', row['Text'])
15     text = text.lower()
16     text = text.split()
17     text = [snowballstemmer.stem(word) for word in text if word not in stop_words]
18     text = ' '.join(text)
19     corpus.append(text)
20
21 corpus[:]
```

Lampiran 3. Kode Program Pembentukan *Word Embedding* dan *Label Encoding*

```
1 from nltk import sent_tokenize
2 from gensim.utils import simple_preprocess
3
4 words=[]
5 for sent in corpus:
6     sent_token=sent_tokenize(sent)
7     for sent in sent_token:
8         words.append(simple_preprocess(sent))
9
10 words
11
12 model=gensim.models.Word2Vec(words,vector_size=150,window=15,epochs=35)
13
14 model.wv.index_to_key
15
16 model.corpus_count
17
18 model.wv.similar_by_word('good')
19
20 model.wv['good'].shape
21
22 def avg_word2vec(words):
23     vectors = [model.wv[word] for word in words if word in model.wv]
24     return np.mean(vectors, axis=0) if vectors else np.zeros(model.vector_size)
25
26 X = []
27 for i in range(len(words)):
28     X.append(avg_word2vec(words[i]))
29
30 X = np.array(X)
31
32 X_train=X[:14123]
33
34 X_train.shape
35
36 X_val=X[14123:]
37 X_val.shape
38
39 X_val
40
41
```

```
1 from sklearn.preprocessing import LabelEncoder
2 encoder = LabelEncoder()
3 y=encoder.fit_transform(training_dataset['Emotion'])
4 y
5
6 label_encoding_table = pd.DataFrame({
7     'Emotion': encoder.classes_,
8     'Encoded Label': range(len(encoder.classes_))
9 })
10
11 label_encoding_table
12
13 y_train=y[:14123]
14 y_train.shape
15
16 y_val=y[14123:]
17 y_val.shape
```


Lampiran 4. Kode Program Tokenizer dan Padding Sequence

```
1 from keras.utils import pad_sequences
2 from keras import Sequential
3 from keras.layers import Dense, SimpleRNN, Embedding, Flatten
4 from tensorflow.keras.callbacks import EarlyStopping
5
6 from tensorflow.keras.preprocessing.text import Tokenizer
7 tokenizer = Tokenizer()
8
9 tokenizer.fit_on_texts(corpus)
10
11 sequences = tokenizer.texts_to_sequences(corpus)
12 sequences
13
14 sequence_lengths = [len(seq) for seq in sequences]
15 maxlen = np.max(sequence_lengths)
16 maxlen
17
18 X = pad_sequences(sequences, maxlen=maxlen)
19
20 X_train=X[:14123]
21 X_val=X[14123:]
22
23 X_train.shape
24
25 from sklearn.preprocessing import LabelEncoder
26 encoder = LabelEncoder()
27 y=encoder.fit_transform(training_dataset['Emotion'])
28 y
29
30 y_train=y[:14123]
31 y_val=y[14123:]
32
33 from collections import Counter
34
35 # Flatten the list of tokenized sequences to a single list of tokens
36 all_tokens = [token for sequence in X for token in sequence]
37
38 # Finding unique tokens
39 unique_tokens = set(all_tokens)
40 print(f"Number of unique tokens: {len(unique_tokens)}")
```

Lampiran 5. Kode Program Pembangunan dan Pelatihan Model

```
1 from tensorflow.keras.optimizers import Adam
2 model = Sequential()
3 model.add(Embedding(input_dim=10087, output_dim=150, input_length=35))
4 model.add(SimpleRNN(32, return_sequences=False))
5 model.add(Dense(4, activation='softmax'))
6
7 model.summary()
8
9 early_stopping = EarlyStopping(
10     monitor='val_accuracy',
11     patience=5,
12     restore_best_weights=True
13 )
14 model.compile(optimizer=Adam(learning_rate=0.0003), loss='sparse_categorical_crossentropy', metrics=['accuracy'])
15 history = model.fit(X_train, y_train, epochs=50, validation_data=(X_val, y_val), callbacks=[early_stopping])
16
17 plt.plot(history.history['accuracy'], color='red', label='train')
18 plt.plot(history.history['val_accuracy'], color='blue', label='validation')
19 plt.legend()
20 plt.show()
21
22 plt.plot(history.history['loss'], color='red', label='train')
23 plt.plot(history.history['val_loss'], color='blue', label='validation')
24 plt.legend()
25 plt.show()
```

Lampiran 6. Kode Program Pengujian dan Evaluasi Model

```
1 def preprocess_text(text):
2     text = re.sub('[^a-zA-Z]', ' ', text)
3     text = text.lower()
4     text = text.split()
5     text = [snowballstemmer.stem(word) for word in text if word not in stop_words]
6     return ' '.join(text)
7
8 # Applying preprocessing to test data
9 test_df['Processed_Text'] = test_df['Text'].apply(preprocess_text)
10
11 X_test = test_df['Processed_Text']
12 y_test = test_df['Emotion']
13
14 test_sequences = tokenizer.texts_to_sequences(test_df['Processed_Text'])
15 X_test = pad_sequences(test_sequences, maxlen=maxlen)
16
17 y_pred = model.predict(X_test)
18
19 y_pred_labels = np.argmax(y_pred, axis=1)
20 y_test_encoded = encoder.transform(test_df['Emotion'])
21 from sklearn.metrics import classification_report
22
23 print(classification_report(
24     y_test_encoded,
25     y_pred_labels,
26     target_names=encoder.classes_
27 ))
28 accuracy = accuracy_score(y_test_encoded, y_pred_labels)
29 print(f"Test set accuracy: {accuracy}")
```

Lampiran 7. Kode Program Analisis Pengaruh Panjang Teks

```
1 test_analysis = pd.DataFrame({
2     'text': test_df['Text'],
3     'actual': y_test,
4     'predicted': y_pred_emotions
5 })
6
7 test_analysis.head()
8
9 test_analysis['word_count'] = test_analysis['text'].apply(
10     lambda x: len(str(x).split())
11 )
12
13 test_analysis.head()
14
15 def text_length_category(word_count):
16
17     if word_count <= 10:
18         return 'Short'
19
20     elif word_count <= 26:
21         return 'Medium'
22
23     else:
24         return 'Long'
25
26 test_analysis['length_category'] = test_analysis['word_count'].apply(
27     text_length_category
28 )
29
30 test_analysis.head()
31
32 from sklearn.metrics import accuracy_score
33
34 results = []
35
36 for category in ['Short', 'Medium', 'Long']:
37
38     subset = test_analysis[
39         test_analysis['length_category'] == category
40     ]
```

```

1 accuracy = accuracy_score(
2     subset['actual'],
3     subset['predicted']
4 )
5
6 results.append([
7     category,
8     len(subset),
9     round(accuracy * 100, 2)
10 ])
11
12 results_df = pd.DataFrame(
13     results,
14     columns=[
15         'Text Length',
16         'Total Data',
17         'Accuracy (%)'
18     ]
19 )
20
21 results_df
22
23 import matplotlib.pyplot as plt
24
25 plt.figure(figsize=(6,4))
26
27 plt.bar(
28     results_df['Text Length'],
29     results_df['Accuracy (%)']
30 )
31
32 plt.ylabel('Accuracy (%)')
33 plt.xlabel('Text Length')
34 plt.title('Accuracy Based on Text Length')
35
36 plt.show()

```

Lampiran 8. Kode Program Implementasi Streamlit

```
1 import streamlit as st
2 import pandas as pd
3 import numpy as np
4 import pickle
5 import re
6 from nltk.corpus import stopwords
7 import nltk
8 from nltk.tokenize import word_tokenize
9 from nltk.stem import SnowballStemmer
10 from keras.utils import pad_sequences
11 from tensorflow.keras.models import load_model
12
13 st.set_page_config(
14     page_title="TEC",
15     page_icon="🤖",
16     layout="centered"
17 )
18
19 with st.sidebar:
20
21     st.markdown(
22         "<h1 style='text-align: center;'>🤖 TEC</h1>",
23         unsafe_allow_html=True
24     )
25
26     st.markdown("---")
27
28     st.markdown("### 📄 About")
29
30     st.markdown(
31         """
32         <div style="
33             background-color:#1e1e1e;
34             padding:15px;
35             border-radius:10px;
36             color:white;
37             border:1px solid #444;
38         ">
39         <b>Text Emotion Classification (TEC)</b> is an application that classifies text into four emotion categories:
40         <br><br>
41         Joy 😊
42         <br>
43         Sadness 😞
44         <br>
```

```

1      Anger 😡
2      <br>
3      Fear 😨
4  </div>
5  """
6  unsafe_allow_html=True
7  )
8
9  st.markdown(
10     """
11     <div style="margin-top:20px;">
12         <h3>🧠 Model</h3>
13     </div>
14     """
15     ,
16     unsafe_allow_html=True
17 )
18
19 st.markdown(
20     """
21     <div style="
22         background-color:#1e1e1e;
23         padding:15px;
24         border-radius:10px;
25         color:white;
26         border:1px solid #444;
27     ">
28     Algorithm: <b>Simple RNN</b><br>
29     Sequence Length: <b>35</b><br>
30     Classes: <b>4</b>
31     </div>
32     """
33     ,
34     unsafe_allow_html=True
35 )
36
37 st.markdown("---")
38
39 st.markdown(
40     """
41     <div style="
42         text-align:center;
43         color:gray;
44         font-size:13px;
45         margin-top:20px;
46     ">

```

```

1         Developed by <b>M. Nizar Asykary</b><br>
2         Department of Informatics<br>
3         Universitas Syiah Kuala • Banda Aceh<br>
4         2026
5     </div>
6     """
7     unsafe_allow_html=True
8 )
9
10 st.title("🧠 Text Emotion Classification")
11
12 st.markdown(
13     "Analyze emotions in text and classify them into **Joy, Sadness, Anger, or Fear**."
14 )
15 )
16
17 st.markdown("****Only support English language****")
18
19 with st.form("emotion_form"):
20
21     input_text = st.text_area(
22         "Enter your Text here"
23     )
24
25     col1, col2, col3 = st.columns([2,1,2])
26
27     with col2:
28         submit = st.form_submit_button(
29             "Submit",
30             use_container_width=True
31         )
32
33 if submit:
34
35     with st.spinner("🔄 Analyzing emotion..."):
36
37         # Remove stopwords
38         stop_words = stopwords.words('english')
39
40         snowballstemmer = SnowballStemmer('english')
41         def preprocess_text(text):
42
43             text = re.sub('[^a-zA-Z]', ' ', text)
44             text = text.lower()
45             text = text.split()
46             text = [snowballstemmer.stem(word) for word in text if word not in stop_words]
47             return ' '.join(text)
48         model = load_model('emotion_classification_rnn_final.h5')

```



```

1      with open('tokenizer_final.pkl', 'rb') as file:
2          tokenizer = pickle.load(file)
3
4      with open('label_encoder_final.pkl', 'rb') as file:
5          encoder = pickle.load(file)
6
7      original_text = input_text
8
9      input_text = preprocess_text(input_text)
10     test_sequences = tokenizer.texts_to_sequences([input_text])
11     maxlen = 35 # Ensure this matches the maxlen used during training
12     X_test = pad_sequences(test_sequences, maxlen=maxlen)
13
14
15     # Make predictions
16     y_pred = model.predict(X_test)
17     y_pred_labels = np.argmax(y_pred, axis=1)
18
19     predicted_emotion = encoder.inverse_transform(y_pred_labels)
20
21     emotion_icons = {
22         "anger": "😡",
23         "fear": "😨",
24         "joy": "😄",
25         "sadness": "😞"
26     }
27
28     emotion = predicted_emotion[0]
29
30     emotion_colors = {
31         "anger": "#e74c3c",
32         "fear": "#f1c40f",
33         "joy": "#2ecc71",
34         "sadness": "#4b5563"
35     }
36
37     emotion_text_colors = {
38         "anger": "white",
39         "fear": "black",
40         "joy": "white",
41         "sadness": "white"
42     }

```

```

1      st.markdown(
2          f"""
3              <div style="
4                  background-color:{emotion_colors[emotion]};
5                  padding:15px;
6                  border-radius:10px;
7                  text-align:left;
8                  font-size:20px;
9                  font-weight:bold;
10                 color:{emotion_text_colors[emotion]};
11                 margin-bottom:10px;
12             ">
13                 Prediction Result: {emotion.capitalize()} {emotion_icons[emotion]}
14             </div>
15             """,
16             unsafe_allow_html=True
17         )
18
19     with st.expander("📊 View Emotion Probabilities"):
20
21         emotion_labels = encoder.classes_
22
23         emoji_dict = {
24             "anger": "😡",
25             "fear": "😱",
26             "joy": "😄",
27             "sadness": "😞"
28         }
29
30         color_dict = {
31             "anger": "#ff4b4b",
32             "fear": "#f1c40f",
33             "joy": "#2ecc71",
34             "sadness": "#2d3436"
35         }
36
37         # Combine labels dan probabilities
38         emotion_scores = list(zip(emotion_labels, y_pred[0]))
39
40         # Sort from largest to smallest probability
41         emotion_scores = sorted(
42             emotion_scores,
43             key=lambda x: x[1],
44             reverse=True
45         )

```

```

1     for emotion, score in emotion_scores:
2
3         icon = emoji_dict.get(emotion.lower(), "?")
4         color = color_dict.get(emotion.lower(), "#3498db")
5
6         st.write(f"**{icon} {emotion.capitalize()}** : {score*100:.2f}%")
7
8     st.markdown(
9         f"""
10         <div style="
11             width:100%;
12             background-color:#e0e0e0;
13             border-radius:10px;
14             height:20px;
15             margin-bottom:15px;
16         ">
17             <div style="
18                 width:{score*100:.2f}%;
19                 background-color:{color};
20                 height:20px;
21                 border-radius:10px;
22             ">
23             </div>
24         </div>
25         """,
26         unsafe_allow_html=True
27     )
28
29     with st.expander("📄 View Preprocessed Text"):
30
31         st.code(input_text, language=None)

```